

Replication Package for Seeing Is Not Complying: Intentional Architectural Erosion under Time Pressure despite Continuous Dependency Visualization

Babette Bækgaard
IT University of Copenhagen
Copenhagen, Denmark
babb@itu.dk

Lotte Juul Damgaard
IT University of Copenhagen
Copenhagen, Denmark
loda@itu.dk

Mircea Lungu
IT University of Copenhagen
Copenhagen, Denmark
mlun@itu.dk

This document is the replication package accompanying the paper “Seeing Is Not Complying: Intentional Architectural Erosion under Time Pressure despite Continuous Dependency Visualization”. It collects the experiment materials and supporting data underlying the study: participant demographics and self-rated C# proficiency (Appendix A–B); the full task-duration distribution and the complete Spearman correlation results (Appendix C–D); the onboarding script and the group-discussion guide; the CoDepend README and delivery guide given to participants; the pre- and post-experiment questionnaires; the task descriptions; the thematic-analysis coding frame and theme summary; and the seven anonymized, English-translated group-discussion transcripts. All participant identifiers are pseudonyms. ArchLens, the visualization studied in the paper, is open source.¹

This package is archived on Zenodo (DOI: to be assigned on acceptance).

¹<https://github.com/archlens/ArchLens>

APPENDIX

A. Participant Demographics

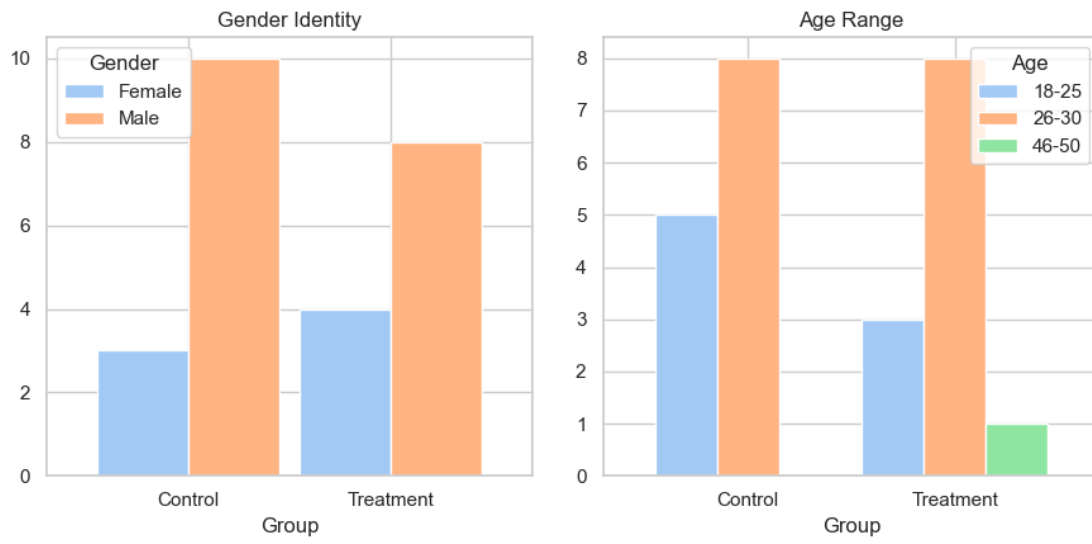


FIGURE I
PARTICIPANT DEMOGRAPHICS.

B. C# Proficiency

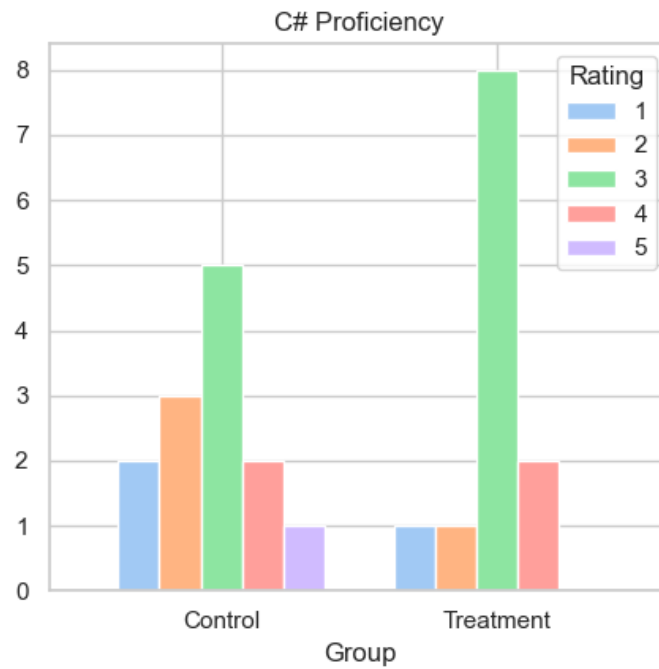


FIGURE II
SELF-RATED C# PROFICIENCY.

C. Task Duration Distribution

		TASK DURATION					
	Task	Count	Mean	Std	25%	50%	75%
C	1	11	47.81	28.15	30.62	45.03	68.14
	2	12	26.20	14.68	17.42	21.98	27.98
	3	10	22.38	12.84	16.07	19.69	26.95
	4	10	19.54	8.55	14.08	20.51	23.84
	5	7	21.48	11.68	13.60	19.17	28.94
	TOTAL	50	28.20	19.66	16.07	22.92	32.65
T	1	9	61.51	23.49	35.93	73.97	78.65
	2	10	29.24	25.46	11.62	16.78	38.87
	3	4	10.36	2.98	8.56	10.74	12.54
	4	4	6.67	1.93	5.62	5.86	6.91
	5	2	14.46	5.43	12.54	14.46	16.38
	TOTAL	29	32.52	28.83	10.62	18.30	69.80

TABLE I
STATISTICS OF TASK IMPLEMENTATION DURATION OVER PER TASK PRs, BY GROUPS CONTROL (C) AND TREATMENT (T). ONLY INCLUDING COMPLETE PRs.

D. Full Correlation Results

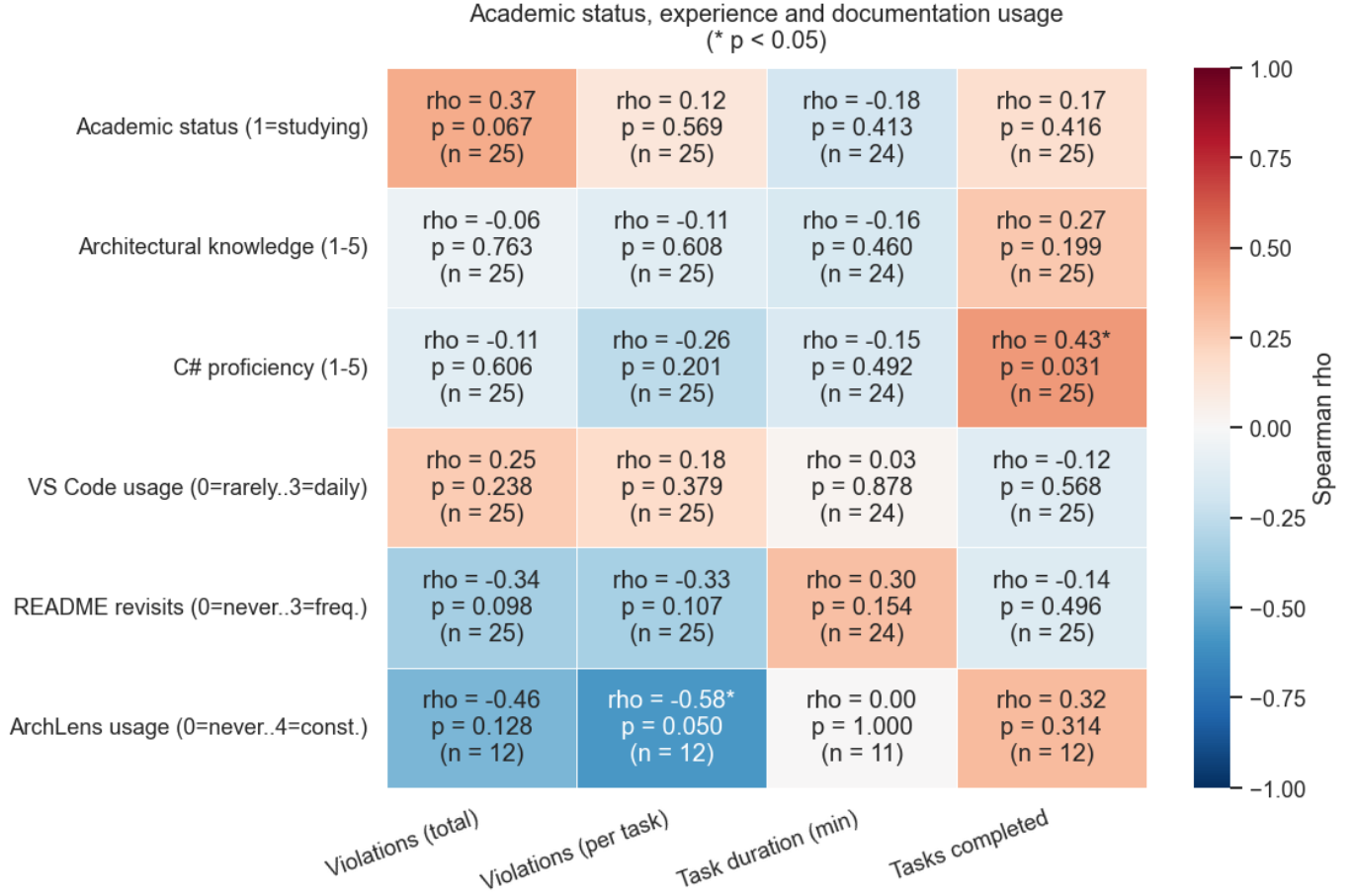


FIGURE III
SPEARMAN RANK CORRELATIONS BETWEEN ACADEMIC STATUS, EXPERIENCE AND BEHAVIORAL VARIABLES AND THE PRIMARY OUTCOMES.
ASTERISKS DENOTE $p < 0.05$. ARCHLENS USAGE IS RESTRICTED TO THE TREATMENT GROUP.

E. Experiment Introduction & Onboarding

1) *Introduction*: Hello, and welcome to this session. We are incredibly grateful for your time to help us.

What will happen today is that you will first answer a survey about your experience, then you will be onboarded to the system you will then implement some tasks for, then you will answer another survey and then we will have a joint discussion afterwards.

So, the plan for today looks like this:

- A short survey about your background (5-10 min)
- Read the documentation and familiarize yourself with the project (40 min)
- Implement the tasks (1 hour 30 min)
- A short survey about your experience (5-10 min)
- A joint discussion at the end (20 min)

We will record the joint discussion on video, but this is solely to make it easier for us to transcribe — the video will be deleted afterwards. We have provided some refreshments, which you are welcome to take as you go. Questions about the overall plan?

2) *Onboarding*: First, let's see if everyone has the things you need installed. I'll read the things out loud and then you can tell me if there's anything you don't have:

- Visual Studio Code
- Python 3.10
- .NET 9
- Extensions for VSCode: C# and C# Dev kit
- (Treatment: Archlens for VSCode)

You're now going to develop in the program CoDepend. You're going to develop in Visual Studio Code, but you're welcome to use tools from other places. You have 1 hour and 45 minutes to complete the tasks. We'll keep an eye on the time and say when the time is up.

Remember that this is not a test of you as developers but of our project. You just have to do what you would normally do. We'll be looking over your shoulder and taking notes along the way. You're welcome to talk to each other during the experiment, but NOT about tasks or the code in general. We'd also like to ask you to avoid looking at what others are doing.

To start, you need to clone the repo into your VSCode. The link is here (on projector). Raise your hand when you have cloned the repo.

In the repository, there are two files that you need to read before you get started:

`how_to_handin.md` — describes your workflow: how you branch, commit and submit.

`README.md` — introduction and documentation for the project itself.

Read both files completely and make sure you understand them — there is important information in both of them, so if there is something you do not understand, ask us. You will find the tasks as issues in the associated GitHub project, but they are also referenced in the documentation. Please wait to look at the tasks until everyone has read and understood the two files.

Raise your hand when you have read both files. We will start the time when everyone is ready.

3) *Treatment only*: You will need ArchLens in this project, it is a tool that visualizes internal dependencies in your program. You can run the script `setup_archlens` in a git bash terminal to have it set up automatically. Otherwise, you will need to:

- Create venv with python 3.10 (Python: Create environment)
- Run `pip install archlens` in a terminal where venv is enabled

You will also need to download the extension ArchLens for VS Code. Let me know when you have everything set up.

...

Here you can see ArchLens. We can see folders and their dependencies on each other. It is interactive and you can click on an arrow to see where the dependencies are. We have defined two different views that you can look at, but the most important one is probably `completeView`. You can also compare your solution with the main branch.

You open ArchLens by running `ArchLens: Open Graph` from the command palette.

We expect you to use ArchLens while you are developing. You decide how you want to use it, but you have to use it somehow.

Questions about ArchLens?

F. Group Discussion Questions

Please note that the group discussions were semi-structured, meaning these questions were a guide - not a concrete list to follow.

1) Control Group:

- 1) What are your overall thoughts about the tasks you just finished? (Were they difficult, easy, confusing?)
- 2) Walk us through how you approached the dependency policy — was it easy to keep in mind while coding?
- 3) How did you check whether your solution conformed to the architecture? (do not say unless anybody says something: What did you refer to?)
- 4) What would have made it easier to follow the dependency rules?
- 5) Any other comments?

2) Treatment Group:

- 1) What are your overall thoughts about the tasks you just finished? (Were they difficult, easy, confusing?)
- 2) Walk us through how ArchLens factored into your workflow — when and why did you open it?
- 3) Were there moments where you chose not to use it, or where it didn't help? What was happening then?
- 4) Did it change how you thought about the dependency policy, or did it mostly confirm things you already knew?
- 5) Would you use something like this in a real project? What would need to change for that?
- 6) Any other comments?

experiment_repos\CoDepend-T\README.md

CoDepend

Make sure to read the the `how_to_deliver_tasks` before reading this file.

CoDepend is a developer tool that parses source-code dependencies and generates architectural visualisations. It runs as both a CLI tool and a VS Code extension, supporting C#, Java, Kotlin, and Go projects.

Getting CoDepend

Currently, CoDepend is under development and not yet published. Therefore the tool is not yet available to use.

Getting Started

Prerequisites

- [.NET 9.0 SDK](#)
- [Visual Studio Code](#)
- C# & C# Dev Kit extension for VS Code

Building the Project

```
dotnet build ./CoDepend
```

Running Tests

```
dotnet test ./CoDepend
```

Running the CLI

```
dotnet run --project ./CoDepend
```

format	diff
"puml" (default)	false (default)
"json"	true

The tool reads an `codepend.json` configuration file that specifies the target project, output format, and dependency views. See `codepend.json` in the repository root for an example.

Project Overview

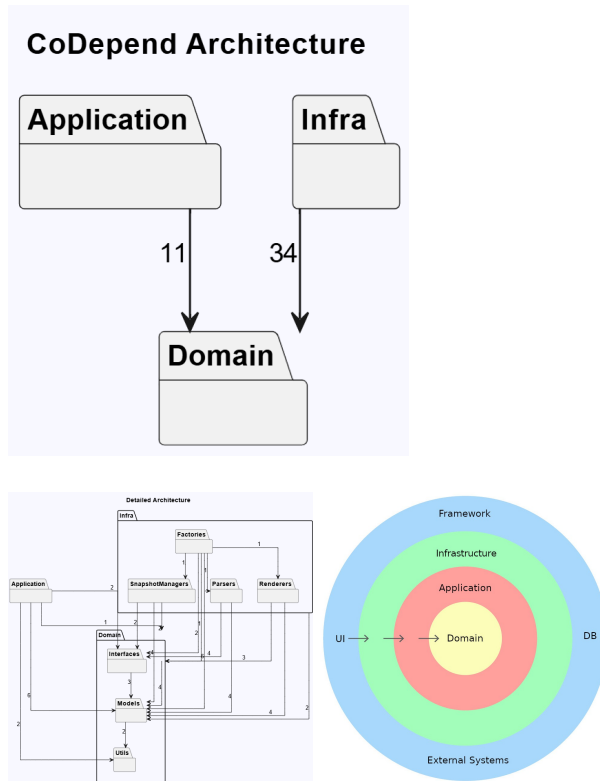
The tool works in three stages:

1. Scanning: The file system is scanned for source files matching configured extensions. A change detector compares the current state against a saved snapshot to determine what has changed since the last run.
2. Parsing: Language-specific parsers extract `using/import` statements from changed files to build a dependency graph of internal project references.
3. Rendering: The dependency graph is rendered into views (PlantUML or JSON), filtered and grouped according to user-defined view configurations.

A snapshot of the dependency graph is persisted after each run, enabling incremental updates and diff-based visualisations that highlight structural changes between the local and remote codebase.

Architecture

The codebase follows a layered architecture with three main layers and a thin CLI entry point:



Domain contains the core models and logic that are independent of any external framework or I/O concern. **Application** contains the use cases that orchestrate the tool's workflow. **Infra** contains all implementations that deal with external concerns / changing environments. **Program.cs** is the CLI entry point.

Dependency Policy

The project maintains a strict dependency direction between layers. **Domain** can not depend on anything outside its layer. **Application** can only depend on Domain and itself. **Infra** can depend on both Domain and Application. **Program.cs** can depend on everything. Dependencies always flow inward.

Please ensure that introduced changes, follows this dependency policy.

Using ArchLens for development

This project uses ArchLens to ensure that the solution complies with the described architecture and dependency policies (described in the section above).

Setup virtual environment

Run the setup script from the repository root:

Windows:

```
./setup_archlens.sh
```

Mac/Linux:

```
source ./setup_archlens.sh
```

OBS! This might take a while, however less than 5 minutes. This creates a Python 3.10 virtual environment in `./venv` and installs ArchLens. Once it completes, activate the environment:

Windows:

```
./venv/Scripts/activate
```

Mac/Linux:

```
source ./venv/bin/activate
```

Installing ArchLens VS code extension

Please install the Archlens VS Code Extension by opening the visual studio code extensions tab, search for ArchLens and install the ArchLens for VS Code.

Viewing visualisations

To see visualised architecture / dependency policy created by ArchLens, open the visualisation, press `ctrl + shift + p` (windows) or `cmd + shift + p` (mac), type ArchLens and select ArchLens: Open Graph (it might take up to 10 seconds before the graph appear).

If the graph doesn't appear or there is an error that pops up, then it is probably because your python interpreter is set to be other than 3.10. Try running ArchLens: Open Setup, then you should get the option to easily switch python interpreter.

To compare the branch dependencies with main, open the ArchLens visualisation window again and check on the `remote` checkbox in the top right corner.

Configuration

The `codepend.json` file at the repository root configures the tool. Key fields:

Field	Description
<code>name</code>	Project name used in output filenames
<code>rootFolder</code>	Path to the source root, relative to the config file
<code>fileExtensions</code>	Which file types to scan (e.g. [".cs"])
<code>exclusions</code>	Directories or patterns to skip
<code>views</code>	Named view configurations with package paths and depth
<code>saveLocation</code>	Where rendered output files are written

Views control what appears in the generated diagrams. Each view specifies which packages to include, at what depth, and which packages to ignore.

Contributing

When working on this codebase, please keep the following in mind:

- **Run the tests** before and after your changes: `dotnet test` from the root folder.
- **Follow existing code conventions:** the project uses primary constructors, file-scoped namespaces, and records where appropriate.
- **Place new code in the appropriate layer** based on what it does, not where it is convenient to put it. If you are unsure, look at how similar functionality is organised in the existing code.

Tasks

See the project `CoDepend` connected to this repo for tasks that needs to be implemented.

License

CoDepend uses MIT Licence.

experiment_repos\CoDepend-C\how_to_deliver_tasks.md

How to Deliver Tasks

Welcome to the repository.

Please read through the whole document before you begin.

Setup

0. Make sure you have Python 3.10 installed in your global environment.
 - Check by running `python --version` in your terminal.
 - If you do not have Python 3.10, download it from python.org.
1. Make sure you have .NET 9.0 or above installed in your global environment.
 - Check by running `dotnet --version` in your terminal.
 - If you have an earlier version than .NET 9.0, update it from [Microsoft: Update development environment](#)
 - If you do not have .NET 9.0, install it from [Download .NET 9.0](#)
2. Clone this repository to your computer and open it in Visual Studio Code.
3. Make sure IntelliSense works. It should if you have the following VS Code extensions installed:
 - **C#** (by Microsoft)
 - **C# Dev Kit** (by Microsoft)
4. Check out your dedicated trunk branch. It is called `<your_alias>-trunk` (e.g. `c0-trunk`). This is your personal workspace — you will only create pull requests into this branch, never into `main`.

Finding the Tasks

From the repository's root page on GitHub, click **Projects** in the top menu bar. Open the project called **CoDepend**. You should see 6 tasks listed. These are the tasks you need to implement. Please complete them in order from the smallest issue number to the largest.

Working on a Task

Repeat the following cycle for each task:

1. Make sure you are on your own TRUNK branch (it will be named with your alias prefixed to `trunk`, e.g. `c0-trunk`).
2. Create a new branch from TRUNK, named with your alias prefix and the issue number (e.g.

c0-1).

3. Implement the task and all related subtasks. A task is complete when you have fulfilled every acceptance criterion listed in the issue, the code compiles, and all tests pass.
4. Create a Pull Request to merge your task branch into your TRUNK branch. **Do not merge the PR and do not request a review** — simply leave it open.
5. Switch back to your TRUNK branch and start the next task from step 1.

Important: Each task must branch off your dedicated TRUNK, not off a previous task branch.

Important: Create the PR as soon as you are done with the task - do not implement all the tasks and then first create the PR's near the end.

The goal is to implement as many tasks as possible. You decide the quality/performance trade-off.

If You Run Out of Tasks

Make sure all your completed tasks have open PRs to your TRUNK branch. Then close the laptop and wait for the rest of the group to finish. We will do the surveys and focus group when everyone is done or when time is up.

If You Get Stuck

If you get stuck on a task, skip it and move on to the next one - If you have partially implemented tasks create a **draft** where you write in the description how many subtasks you implemented (e.g. if the task has 5 subtasks and you have implemented 3, create a **draft** with the description "3 subtasks out of 5").

If you ask for help, we might choose not to assist you, as our guidance might bias your solution. If you believe an issue description or a test is broken or not described properly, you may flag it to us — but we cannot help with implementation or describe the task further for you.

Breaks

You manage your own time. If you need to go to the bathroom or similar, you do not need to ask for permission. You may also talk to the other participants, but **not about the tasks or how to complete them**. Your completion time is recorded, so please be mindful of that.

I. Pre-Experiment Survey

1. Please provide your name and email (e.g. "Babette (babb@itu.dk)")
2. Please provide a link to the github account that you will use for the experiment
3. How old are you?
 - 18-25
 - 26-30
 - 31-35
 - 36-40
 - 41-45
 - 46-50
 - 51-55
 - 56-60
 - 60+
4. What is your gender identity?
 - Male
 - Female
 - Non-binary
 - Other/Won't share
5. How many years of programming experience do you have? (Including studies, professional and hobby projects)
6. How many years of professional programming experience do you have? (Only full-time jobs)
7. How would you rate your proficiency with C#? (1-5)
8. How often do you use VS Code?
 - Daily
 - Weekly
 - Occasionally
 - Rarely
 - Never used it
9. What is your degree? (e.g. "MSc. Computer Science at ITU", "BSc. Software Development at ITU", "Professional Bachelor Software Development at DTU", etc.)
10. Are you currently studying or have you completed this degree?
 - Studying
 - Completed
11. What is your current position? If multiple, separate by ';'. (e.g. "IT Consultant", "Between jobs")
12. Are you full-time or part-time?
 - Full-time
 - Part-time
13. Consent This study consists of two parts: a set of development tasks and a group interview. The purpose of the study is to assess the effectiveness of a developer tool. In the first part, the code you write will be temporarily linked to your email address in order to match it with your survey responses. This information will be anonymized before analysis. In the second part, the group interview will be video recorded for transcription purposes only. The recordings will only be accessed by the research team and will not be shared outside the project. All recordings and personal data will be anonymized. Participation is voluntary. You may withdraw from the study at any time and request that your data be deleted, unless the data has already been anonymized. Data will be stored securely and retained until 1/7 2026, after which it will be deleted. If you have any questions, please ask us now or contact us via e-mail. By participating, you confirm that you have read and understood this information and consent to take part in the study.

Thank you, Babette Bækgaard (babb@itu.dk) and Lotte Damgaard (loda@itu.dk)

[] I understand and consent to the conditions

J. Post-Experiment Survey

1. Please provide a link to the github account that you have used for the experiment
 2. How would you rate your software architectural knowledge? (1-5)
 3. How familiar are you with clean architecture principles? (1-5)
 4. Have you worked on a project where architecture was an important part of the development? (Yes/no)
 5. If your previous answer was yes, please describe how, in with context and what architecture was in place
 6. Have you used any architecture or dependency visualisation tools before? If so, which?
 7. How difficult did you find the tasks overall? (1-5)
 8. How confident are you that your solution conforms to the dependency policy/architecture introduced? (1-5)
 9. Did you refer to the architecture documentation during the task? (Here we mean the static documentation in the README, not the interactive visualisation provided by ArchLens)
 - Never
 - Once or twice
 - Several times
 - Frequently
 10. How often do you use AI to code ?
 - Never
 - Yearly
 - Monthly
 - Weekly
 - About once a day
 - Multiple times a day
 11. Did you use AI during the task? If yes, please explain which AI model (Claude code, ChatGPT, etc.) and for what purpose. If possible, copy in your prompts.
 12. We would like to attribute you in some way for helping us - are there any of these options we can do that?
 - No thank you
 - Tagged in a LinkedIn post
 - By name in the Acknowledgement section of the report
 - Other
 13. Any other comments?
- Treatment group only**
- T1. Did you use ArchLens during the task?
 - Not once (never)
 - 1-2 times (rarely)
 - 2-4 times (occasionally)
 - 4-6 times (regularly)
 - I had it open all the time (constantly)
 - T2. How did you use ArchLens?
 - T3. Did ArchLens help you identify any dependency violations?
 - T4. Did ArchLens help you avoid any dependency violations?
 - T5. What was most useful about ArchLens?
 - T6. What was most frustrating or unhelpful about ArchLens?

K. Task Descriptions

1 - Logging

1a Implement a `Logger` class in the `Infra` folder with three methods that format and print the input to the console. `LogInformation`, `LogWarning` and `LogError`. The methods do not return anything, only prints the input.

Acceptance criteria:

- There is a `Logger` class in the `Infra` folder (remember to add correct namespace)
- There is a `LogInformation` method taking in a string, printing "INFO: {input}"
- There is a `LogWarning` method taking in a string, printing "WARN: {input}"
- There is a `LogError` method taking in a string, printing "ERROR: {input}"

1b Use the logger to log the size of the result of running serialization in the `DependencyGraphSerializer`.

Acceptance criteria:

- When serialization finishes, before the result is returned, the size of the `byte[]` is logged at information level
- If the `byte[]` is empty, it is logged at warning level instead

1c Use the logger instead of `Console.Error` in `DependencyGraphBuilder`.

Acceptance criteria:

- In `TryClassifyItem` the logger logs the exception at error level
- In `ParseFileItemAsync` the logger logs the exception at error level

1d Use the logger to log at information level whether `UpdateGraphUseCase` is running with diff or not.

Acceptance criteria:

- `UpdateGraphUseCase` logs "Running diff use case" if diff is true
- `UpdateGraphUseCase` logs "Running non-diff use case" if diff is false

2 - Exclusions

2a All the logic for exclusions that is currently in the `ChangeDetector` class should be moved to it own class called `ExclusionManager`.

Acceptance criteria:

- There is a class called `ExclusionManager` (remember to add correct namespace)
- The class contains the `ExclusionRule` record
- The class contains public methods:
 - `IsExcluded`
 - `NormalizeExclusionEntry`
 - `CompileExclusions`
- The class contains private methods:
 - `MatchesSuffixPattern`
 - `ToDirPrefix`
 - `GetRelative`
- All methods are static

3 - Repository

3a Implement a basic `Repository` class, it should be in `Infra` and contain the methods `ProjectDependencyGraph? GetSnapshot()` and `void SetSnapshot(ProjectDependencyGraph snapshot)`. For now, `GetSnapshot` should always return null.

Acceptance criteria:

- There is a `Repository` class
- There is a `GetSnapshot` method that returns null
- There is a `SetSnapshot` method that takes in a `ProjectDependencyGraph` and currently does nothing

3b Use the previously implemented `GetSnapshot` in the `UpdateGraphUseCase`, such that it uses the repository snapshot if it is not null, otherwise calls the `SnapshotManager`

Acceptance criteria:

- In `UpdateGraphUseCase`, `GetSnapshot` is called
- If `GetSnapshot` returns null, call `SnapshotManager`
- For testing reasons, it should be possible to run the use case without a repository

4 - `RendererBase`

4a The `RendererBase` should normalize the `FileExtension` before using it. Beware of correct dots. See `ConfigManager.NormalizeExtension(ext)`

Acceptance criteria:

- `RendererBase` normalizes the `FileExtension`

5 - `ConfigManager`

5a Instead of passing all the configs around, the `ConfigManager` should contain query methods for getting the different options.

Acceptance criteria:

- There is a `GetBaseOptions` method
- There is a `GetParserOptions` method
- There is a `GetRenderOptions` method
- There is a `GetSnapshotOptions` method
- The getters throw an exception if `LoadAsync` has not been run yet (and fields therefore are null)
- `LoadAsync` still returns the options, but only for the unit tests to work

5b In `Program`, the `ConfigManager` is initialized and loaded. Factories receive the options they need using query methods, while the use case should take the whole config manager as a parameter.

Acceptance criteria:

- `Program` loads the config
- All three factories receive their options from config manager query methods
- The `UpdateGraphUseCase` receives the config manager as a parameter, and queries for the options when needed

5c The loading of the configs should no longer be in the `ConfigManager` itself, but in a new `LoadConfigUseCase` that initializes the `ConfigManager`.

Acceptance criteria:

- There is a `LoadConfigUseCase` with a `RunAsync` method
- The use case contains all the logic for loading the config from the file, including relevant DTO's
- `RunAsync` initializes and returns the `ConfigManager`
- The `ConfigManager` now takes the four options as parameters in the constructor
- `Program.cs` runs the use case

Code name	Definition
AI	
AI and docs	What documentation is AI being given?
AI as an assistant	How is AI being used during dev?
QA of AI code	How to ensure AI code is good? (Using ArchLens to check)
Func. vs non-func. req	
Speed vs. quality	How important are soft req. when there is a deadline?
Prioritizing architecture	When is arch. prioritized?
ArchLens usage	<i>(Treatment only)</i>
UI	Comments on the UI of ArchLens
Speed	Comments on the speed of ArchLens
Understanding of codebase	
Architectural awareness	Where they aware of the arch. of the codebase?
Code comprehension	Did they understand the code?
Onboarding	Comments on the onboarding to the system
Documentation (static)	
README	Comments on if and when they used the README
Rules	
Enforcing or not	Should rules be enforced or simply warnings?
Architecture in general	
When/pipeline	When is arch. made/updated?
Importance	When is arch. important? When is it not?

26-05-2026 15:12

Theme Summary

Thesis - babb & Ioda

Codes\Themes\1-func-non-func

Files\0904-m-c Control

Number Of References: 2

For me it's something I think about when I realize that this isn't working very well. And then you think, what kind of architecture could improve this. And then you start moving things around and stuff like that. But it's kind of something I usually think about afterward. While there are perhaps many who do it the other way around. They start by thinking, "This is the architecture we're using," but I think I tend to find the architecture that makes sense based on what I've already built.

I also think that if you just try to follow a predefined architecture, it's like trying to force something into a template

Files\1304-e-t Treatment

Number Of References: 5

If there was an arrow that wasn't supposed to be there. Then you went back and fixed it. Or at least that's what I did... It wasn't that good, move on.

So if you use these templates, these architectures that you know, it's much easier for new developers who come in, and then you know how it works in some way. And then you can also make sure that you don't do some completely crazy things when you have these rules that we follow in the company, and everyone knows the same thing.

I was so busy.

And you really have to, you really shouldn't trust that just because it has controller in the name that that's what it does. So that makes it a little more complicated, I would say. So it would have been nice if they had stuck to the rules that are in place. Yeah.

Do you think it's important to adhere to the architecture of projects?

[14:15 - 14:17] SPEAKERS: Yes... (muffled agreement)

Files\1404-e-c Control

Number Of References: 12

Do I know it's something that's going to be big, or something that is big? So for me, yes, it's important. If it's something small, today at least, especially with AI, then it's like, no, not really, it can be fixed quickly. That's also maybe a naive way of approaching it.

So in the end, when it comes to a specific day to day like this, we try to do it as quickly as possible. And then we don't focus so much on the quality of the code. But you want to. And it's a bit annoying sometimes.

So the difference is maybe that when we actually have these very specific tasks, it's a little easier to follow and conform to the rules that are there. But at the same time, I have no idea what it's going to be used for.

So it is very important that the structure is complete to a certain extent. And then you can move around if you need to change something. So I always keep that in mind when I code, and try to keep track of the right things in the right places and handlers and models etc. There is such a very strong structure that we follow.

I think for myself I like to keep a fair amount of structure, at least I tell myself that I like to actually refactor and keep things clean. But I would say my insight is that that's not what people pay for. People pay for software that works, and if you tell them that we can make it easier to maintain, you get less technical debt, you have to be a good sales person, and there are also some who accept that, but it becomes a trade-off. You have speed... Some of you said earlier that we had a deadline of an hour and a half. So it was like, what do you value more? Is it that you followed the architecture, or is it that you got the features done? So that's the trade-off that you're up against.

But you're also right that people don't pay for cool code, they pay for it to work. And I know that from my work too. It's very difficult to convince a bank that they need to get off the mainframe if what's on the mainframe works. So, that...

I can understand that consultant's brain, where it's like it has to be completed. Then the quality doesn't matter. I've probably screwed up the architecture a bit now and then. So it's a nice challenge.

It went really well at first, and then at some point my brain took over and was like, okay, we're going to get it done. It was like a consultant's brain, that's how we're going to get it all done, and then it just went faster and faster. The quality maybe dropped a little, but good challenges.

No one wants to pay for the extra. It's me... I want to have a better stamp of approval on this, and that's why I spend the extra hours on it. But the thing is, if you want to work 24 hours a day, you can do that too, but you only get paid for what the customer wants to pay for.

At my job, because I sit and work with mainframe code, most of the code I work with is older than the Clean Code book. And much older. There is no real pattern in how to do includes and things.

I think I was like, okay we have an hour and a half, or we have some timeframe, so like, I don't pay too much attention to the details

I also sat and talked about how it's important to have good architecture. But in the end, I don't get paid to write comments or tests.

Files\1404-m-t Treatment

Number Of References: 6

You have lots of reviews, you have lots of functional reviews, but it's really rare that you delve into, how has the architectural quality of what we've done moved things? Have we actually delivered anything where we haven't delivered more hours of work to ourselves down the road?

I think I was influenced by having the visualization that was given to me, so I think I was more influenced by not wanting an arrow pointing the wrong way. It almost became gamified. And of course, since it was yesterday, that lecture, there might be some attention bias. It was very new to me. So fresh in my memory. But it's hard for me to say how much it was.

I also just focused on getting the tasks done, but having visual feedback like that is really good. I just think it's a super good tool. I'm not used to working with development, so I just focus on actually getting something in.

It wasn't like you weren't made aware that right now you were actually on your way to implementing something that you would probably have to redo again at some point, or that would become a bit of technical debt at some point, if it was a bigger thing. So in that regard, the mission was accomplished in terms of the fact that it's not something you... Just missing... Often you have a lot of considerations when you're developing something. So it's not just about conforming to an architecture, it's really about: Does it work. Because you have a PO who's standing there tugging at you and saying: Does it work. Because I'm going to show it in five minutes. So it has to work and it has to be, you know, two seconds before we get that answer, or whatever. And then it's just really nice to have some tools that we agree on. This is not finished until these arrows turn this way, we agree. So, often and often, it is also to visualize these challenges to those who are sitting and are responsible for the business part.

That functionality isn't the only parameter. And that's just... I just think that's how the world is. Somewhere, and you can quickly become a bit of that developer who sits in the corner and snipes at something. Where in reality it's just... These are pretty important things to get right. It's a bit like coming out and saying: I would really like us to fix our legacy code. That's not something that gives promotions, right? It's just when there are layoffs and things like that. Then it's not the people who sit and make the newest product who are smoking. It's the ones who sit and make legacy. You know that sometimes you can be a bit of a minority if you sit and want to make proper architecture. Yes.

That's also this whole AI discussion, that's also really down to this. Yes, it's great, we can make some code quickly. But are you willing to pay what it costs to have fast code? Because all the arrows don't turn right. So it's just a tradeoff. As long as we all agree that it's a tradeoff. And that thing with the visualization of: No, we have something where all the arrows are fucked up. But if we agree that this is what we sell, fine.

Files\1904-m-c Control

Number Of References: 11

I think most people don't want to sit and code review imports. Unless you see that changes have been made up there. Then you just go to the code a little faster.

r01: So the description of the tasks weighs more than the description in the documentation?
[05:19 - 05:21] c10: Yes, I would say that.

At my old job we were fucked. There was nothing. So you know, whatever structure you made was more the structure that was there when you started. So I don't know.

I actively think about it, so now I'm kind of familiar with onion and clean architecture, etc. And it's really something I think about in my work too. So it's a bit like, I want to say an occupational injury, I can't really help it. So I think it's so active that I thought about it, but I also feel like I actively made a choice to compromise with it sometimes.

I've had that feeling too. And then I also wanted to move the small modules around, because then I'm like, no, I can't do that, I don't have time for that.

I also caught myself thinking, this, if I should have done it properly. So I would have spent an hour and a half just on this. Well, but I knew, I'm going to have to, so you just have to... Compromise on some things.

You could also show that, at least I feel that way, to a project manager. Try looking here. If I show you that it says using this, it doesn't mean anything to you. But if I show you in this PR, which you might have a little more understanding of, that there's a big red cross that says NO.

Then you might say, okay let's rephrase it. Because otherwise I feel that if there aren't any checks like that, and it just says green all the way, then that's it, what's the problem?

It depends on the culture you're in. If I was at a workplace where I was told that remember the most important thing is that you keep these architectural things that are in the code in advance. Rather than the tasks you get from, you know, Karsten from the fourth floor, who has never opened a computer himself. Okay, fine. But if it's something like that, where these tasks are very strictly defined by the customer's IT consultants, they just have to be adhered to. And then, whatever documentation, it's just whatever. So that's how it is, so you do it.

It's a bit difficult, because in these tasks sometimes it's impossible. Then it's because I agree that I would talk about finding a way to restructure it, but when you have an hour and a half. It has to be done. Then there's a lot of context within it. Okay, I have time pressure, so it's probably some kind of deadline or busyness. There are times when it's such small tasks that have to be done. It's probably some consultant work. And then it becomes a lot just like that, it doesn't matter. It also comes from my own experience, where it's just fucking architecture at my old job went out the window, who gave a shit.

Obviously, I'm not going to break those boundaries, but I'm putting it in the wrong layer a bit.

We don't have that. No README, no architecture. Only vibes.

Codes\Themes\2-enforcement

Files\0904-m-c Control

Number Of References: 5

It seems to work—I mean, it depends a bit on the size of the project, but it's also something where, if you really want those strict boundaries between things, that's usually the best way, I think. Otherwise, I think you need some kind of code analyzer that checks where namespaces are being used. It's kind of an either-or situation, because if you want to enforce something, you have to enforce it. Otherwise it just becomes a game we're playing, right?

Yeah, it gave me a bit of a headache once. It was kind of that thing you were talking about, where people tried to say, "This is how we do it because that's what people say is the right way to do it," but it just didn't quite fit the project.

I just want to say that I agree too, because discipline is really important when you're working on a codebase. If you don't have that separation—where the technology fails if you don't do it properly—then people are like, "Yeah, we're a bit busy, let's see if we can get around it." If you want to be fast, you just break those rules.

And when is the architecture mature enough that you dare to enforce something? When you're not still in the process of domain-mapping something?

The README mentioned: Here's some architecture we're really focused on—we have to follow this clean architecture. But it doesn't seem like there was anyone who... I mean, sometimes you can add code analysis and things like that to make sure you don't break out of these things, for example

Files\1304-e-t Treatment

Number Of References: 7

It would be great if it was an automatic check when you write the code.

Yes, a linter or something like that

I'm just sitting there thinking fuck, that would be annoying, so when you're sitting there and you have to push something, and then those arrows come up.

I think it would be great if it was part of the suite. I can see that you're saying to a pull request, you can't do that

But I think maybe I could have done it better, because my code might have been a little cleaner. If it had just said no, you're not allowed to do this.

But more as a part of some CI/CD workflow when you push, than just something for a PR, it spits it out for a preview or something, I don't know. So you can sit and watch.

Then it should be part of a workflow like that.

Files\1404-e-c Control

Number Of References: 4

I think, especially when you get tasks like this, the thing with visualizations, I agree, that is also what I started by saying. Even more strongly I would say that in the same way that we have unit tests, if you have architecture tests. Hard limits. If you could set it up, then it is very easy to follow, because then it is just like that...

So you could say that if you have some hard tests or linters, or whatever you're running on this, yeah, and then boom, it's not working.

Yeah, exactly, but you could also say that then you can't ship your feature. And then it would be another place where you could...

Yeah like, you just get a fail. If you try to push something, you have this hard rule. The hardest thing you can do is set up a clean system, just like you can have linters or another system. That also falls into some documentation, they say, you don't document in the code, you document next to it. I think the best thing is as close to the code as you can have it. That would be cool. And then you can also visualize it. I think both things are good to have.

Files\1504-e-t Treatment

Number Of References: 1

And I also think that it will also be easier when you do something like peer review. If someone is doing a PR, then it's much easier to catch. If someone has broken it by doing, that is, running this analysis, and then having it in a pipeline, where a picture just comes up and says, now the structure looks like this. So with green and red arrows you could easily use it to be a bit like, hey, we can see on your PR that you've added this green arrow, so it shouldn't be there. I think that could be useful there. It's just a question of using more as validation that it hasn't broken anything.

Files\1904-m-c Control

Number Of References: 3

If one can abuse it, then you abuse it. So that's just the rule of thumb, so you should almost have an architecture that supports the changes you're working on, because it's going to come through, I feel like saying no, if you've made a bad decision. So I would prefer that.

I would really like it to be like a compiler warnings thing. We started out with 4 compiler warnings, and what if there were 4 compiler warnings that said, you're breaking the architecture here, do you want to continue doing that, because sometimes it might be necessary, because architecture is not a one-to-one absolute truth either. Most of the time it is, but there are some examples where you have to break it a little bit, to support some other good ideas. So it would be really cool to have a built-in architecture analyzer.

I would also say that it's like this... Just in the PR that an automatic message comes up that says... Well, just a, I don't know if you can call it that, or just like checking, and just saying hey, you're no longer following the architecture. It's just a test that fails.

Codes\Themes\3-code-comprehension

Files\0904-m-c Control

Number Of References: 7

The whole process of establishing the domain probably takes longer than when you have to complete some tasks in an hour and a half.

I got a little stuck at the end because I had two things on my mind: what the program does, and then how it's abstracted and structured, and I couldn't really map those two things in my head because I don't know the system.

I also think it's that tricky distinction between what represents real life, where you go out and code something you don't know what it is. Or do you need a more thorough introduction to what the system actually is and what it's supposed to achieve than what's necessarily just in a README. There's also a context around development that can be a bit tricky.

Well, I think that if I'd had a slightly more thorough... where I'd looked around a bit more at what the architecture is, where things are, and what's called what. Yeah, getting that kind of bird's-eye view could have been helpful, I think. I think very visually

I still don't really have a grasp of the entire codebase, and I just completed the tasks

I also think it's a question of how much you're aware of architecture to begin with, in terms of that progression. That, at least, when I did the first assignment, and then did number two, and then when I got to the third, I was like, "Oh man, I should have solved number one differently." Now that I'm looking at the system and have a better grasp of it.

But you can also abstract a domain to the point where it's completely unintelligible. Where you've pushed the abstraction layer so far that you don't really know what the program does. Or people coming in from the outside as new contributors can't really figure it out because it's too complex.

Files\1304-e-t Treatment

Number Of References: 1

I could have used a little more time sitting down and looking at the code before I started the task. And understanding the context.

Files\1404-e-c Control

Number Of References: 2

Where here, I felt a lot that there are these rules, this has to be done, just do it. Hope that what I did followed the rules. And if I have followed the rules, then hopefully it is good enough. I am not used to working that way.

So the difference is maybe that when we actually have these very specific tasks, it's a little easier to follow and conform to the rules that are there. But at the same time, I have no idea what it's going to be used for. I don't understand the design choice at all to make this new logger instead of doing something else. So that might make it a little difficult, when you don't really know what the reason is that we're implementing these things. Because we haven't been involved in a design process, we haven't been the architects of it, we don't know what the architects have in mind. We just carry out the tasks like that. So it can be a bit difficult. Because you don't know if you're following the wish that was 100%, in addition to just following the requirements. But is following the requirements good enough, that can be hard to say.

Files\1404-m-t Treatment

Number Of References: 1

I think they were okay well described, but I couldn't really see if I should think more than that. Because I could clearly see that there were some dependencies, but I ignored them.

Files\1504-e-t Treatment

Number Of References: 4

Yes, I also think that in connection with onboarding, because I'm not familiar with the project, it's a good way for me to become familiar with it and see where the different boundaries are.

And then getting into this, you should first familiarize yourself with what exactly it is, how the structure works. And so I knew that I came in with a mindset that was like, okay I can just add this. And I just couldn't do that at all. And so I was like okay...

I just think it's difficult when you just see something and you have to keep this, and then you're like okay shit, good luck.

The longer you've been on something, the easier it is to have that overview, without having to use any visualization tools

Files\1904-m-c Control

Number Of References: 6

It's so tightly defined that the assumption of, okay, but then I have to move this around, and things like that disappears a bit. Because then it would probably be there. So it suddenly becomes obvious that it's just breaking the architecture.

There were some things where I didn't think it should have been in the layers they were. So I would have probably done a refactoring first. Or asked how we could solve this best.

How do you best maintain the current architecture, because I was... at a point when I started having references from application to infrastructure, where I was like, mmm, I'm cooked. That's probably fine, bang! And then I went back at some point and saw that it probably wasn't fine.

Well, it's a funny architecture. You also have to catch it. And that's exactly the thing about it being clean.

And then my approach is... Damn! Alright, you're the ones paying me, fuck it. And then just go anyway. Because that... Is it stupid? Yes. Is this system... I don't know this system. I don't know, it might be possible. So if this is the task I'm given, then I also assume that it's already

messy enough. If these are the tasks that belong to the system, then the system must have a corresponding level of messiness. So that's why I'm...

So in the other dependency files, infrastructure and application are on the same layer. They just don't point to each other. They just both point to domain, so there is double confusing or incorrect documentation. And that was also something that confirmed for me that this is a messy architecture system. It just doesn't fit. It's some legacy documentation that doesn't work.

Codes\Themes\4-static-docs-arch-awareness

Files\0904-m-c Control

Number Of References: 8

I tried to stick to that onion architecture thing because I'd heard about it in BDSA. So I had it in the back of my mind, but not because I used the README

I also think it's that tricky distinction between what represents real life, where you go out and code something you don't know what it is. Or do you need a more thorough introduction to what the system actually is and what it's supposed to achieve than what's necessarily just in a README. There's also a context around development that can be a bit tricky.

I think there are some projects where they don't write a README. And then there are some where they stick to the classic open source approach, where it's like, "Here's how you run the project." You need these things to run it. And then there are others where they say, "These are our coding standards for this project."

I think I forgot all about it when I started coding.

That made it a little hard to navigate between what it seemed like was mentioned in the README that you were supposed to follow.

I looked at the diagrams in the README, but they didn't speak much to me

At my old job, for example, there was nothing like that at all. With the architecture, it was just a matter of figuring it out.

The README mentioned: Here's some architecture we're really focused on—we have to follow this clean architecture. But it doesn't seem like there was anyone who... I mean, sometimes you can add code analysis and things like that to make sure you don't break out of these things, for example

Files\1304-e-t Treatment

Number Of References: 3

Well, I didn't look at import statements

If it wasn't in there, then I wouldn't have thought about it, I guess. So I would have just... Driven off and just made it work, and not really thought about...

That's what you would do in reality, if you were to contribute to some open source code, it won't be merged if you don't follow the rules that they've set up. Yes.

Files\1404-e-c Control

Number Of References: 5

Well, I think you've described in the README, you've described something, plus the images that aren't there, some kind of clean architecture. You had kind of set some constraints on the domain and application and infrastructure. So it was like, based on that, there's some kind of architecture and the coupling was also, it sounds like clean. You probably had that in your picture too, and it's like, again I think I was like, okay we have an hour and a half, or we have some timeframe, so like, I don't pay too much attention to the details, but that, it sounded like a hard requirement

I probably forgot about it a bit. I want to be honest, I forgot about it. So, yes.

I was also looking at import statements, but I was more focused on the acceptance criteria.

But in hindsight, I should have looked a lot more at import statements. So I looked at that to start with, and I was like, yes okay, you've made a new interface. Boom, it fits with your namespace, domain, fine. But I didn't really do it enough, so I should have done that more.

So the structure should preferably be there to some extent, even if we don't have the strongest rules. When I was hired, no one said that this, this should be here.

Files\1404-m-t Treatment

Number Of References: 1

But it's just as much about adding things, but that it's going the wrong way. Yes, that's illegal. Yes, the rules you had written in that README, that it also showed that.

Files\1504-e-t Treatment

Number Of References: 5

And I just believe in my experience with the documentation that is sometimes created, where it is the case that the documentation is fine. And it is very fine that there is a lot of text. And for example in the README and things like that. But it also just makes it so that when you need some information quickly, it quickly becomes very difficult to find, which I think is why I resort more to ArchLens rather than looking in the README. Because in the README there is some text that takes up space, there is no bloat as such, because it should be there. But when I just need to know if I've violated a violation, I don't want to go through the README with any text. Then I'd rather just pull it up and be like, here's what you've violated, thanks.

I quickly forgot about it. I just created it and out of the corner of my eye you could see that something is wrong over here. I can see I have someone pointing to it there. I know that's not wrong when I look at it. You were reminded of it. I think if you had it closed and didn't see it, I don't think I would have noticed it.

If you could just have the README open next to you, in a location where you know this is where I'm going to look, it goes faster.

Because it said very specifically in the readme that let's try to keep the architecture that is there, so that domain and application can't refer to the infrastructure folder. And it did that for me, so I got the idea that I can't. Or at least, I'll try to avoid that.

Especially because it breaks your flow of what you're doing right now. If you're suddenly in a good coding rhythm, you don't want to go into some long documentation and read it all through, it's better to just have a quick overview.

Files\1904-m-c Control

Number Of References: 13

So in the other dependency files, infrastructure and application are on the same layer. They just don't point to each other. They just both point to domain, so there is double confusing or incorrect documentation. And that was also something that confirmed for me that this is a messy architecture system. It just doesn't fit. It's some legacy documentation that doesn't work.

At my workplace there are a lot of protocols for everything, so I always have open on another screen, okay, next step in my to-do, of getting things into production. So I sit there all the time and look at it.

I think I started by doing things quite manually, also in that logging, that was definitely the one I spent the most time on. Where I could see without using any tools or anything. I can't just call in the wrong direction in relation to this onion structure. But I can't remember how to do it if I have to get around it.

I thought that document with what you did was if you actually wanted to make real changes to your code. And there I was like, I'm not supposed to, I just have to do some tasks. And so I didn't take it that seriously.

I thought those architectures were examples of how you could use your tool. So I was like, that's cool, and I didn't even think that we were supposed to keep it.

I'm sitting with the README on one side of my screen and I can see, well, but I'm not allowed to have an import that way. Then I can see that I need some functionality, like a logger class I just made. Which I'm not allowed to call. And then I talked to an AI about how to get around it. And I ended up having a lot of doubts about how in-depth you should be, how clean it should be.

I sat down and started checking references.

I've used AI for that, where I've kind of fed it the context and the README. I've made a mandate for what architecture it should follow, unfortunately AI isn't that good at following architecture, but it was kind of made aware of, uh, that something is starting to happen with some architecture, so if that's the case, then I have to sit and read references.

We don't have that. No README, no architecture. Only vibes.

It's funny, there are people who have tried to set up some rules at some point, and I can see that they've been thrown out the window.

It's funny because I've had exactly the opposite attitude, and it might also come from the fact that at my job, there are those who are very detail-oriented, because it's something that's very, very important to a lot of people. So the thing about, okay, if it says in the documents, I looked at it many times, like okay, what are the rules? What can I call what? Okay, like that. That was the first priority, so it didn't matter how far I got.

Yes, a few times, but no more than that. 1-2 times.

I mostly looked at that docs folder. Where there were 3 pictures in it.

Codes\Themes\5-ai

Files\1304-e-t Treatment

Number Of References: 11

Also in a time when AI is being used more and more to code, it's nice to say bro, you've fucked up.

And then I think I prompted my Claude. Not Code, but just the regular one. And it totally drove me crazy for an hour. And then I realized it was completely crazy when it asked me to change your test, and then I thought, let's revert back and find a better way, and then it went better.

Now I've also used Claude to solve them, so it was a little more to keep an eye on, okay, are the dependencies so... Which ones have you added? Added the ones that I expected it to add? And did it violate the dependency policy?

Maybe I'm a little too inclined to just trust some AI

But if you could create an abstraction layer that was also more readable for AI, that could kind of... Don't have to read all sorts of files, and all sorts of things that clutter it, if you could just get the pure graph representation spit out and then match it up against a spec or an architecture, where you have to have a convention with a name or something.

I looked at the code to check the acceptance criteria, and then I used ArchLens to check the dependency policy.

I have a plus one on t07, that is. That GitHub Copilot there.

Well, I didn't look at import statements, but it's very nice to have a check like that to make sure Claude didn't hallucinate that it was supposed to import something from something it wasn't supposed to, right?

The thing about there being acceptance criteria. That was good. That's good for an AI

Claude didn't understand it either. The first time. But it did after it was just whipped a little more.

I straight up just copy paste into Claude. Hey Claude, I've got this task. Go. And then it solved it for me.

Files\1404-e-c Control

Number Of References: 10

I primarily kept an eye on it so that it doesn't start importing and using things across the layers in a way that it shouldn't.

Do I know it's something that's going to be big, or something that is big? So for me, yes, it's important. If it's something small, today at least, especially with AI, then it's like, no, not really, it can be fixed quickly. That's also maybe a naive way of approaching it.

But in hindsight, I should have looked a lot more at import statements. So I looked at that to start with, and I was like, yes okay, you've made a new interface. Boom, it fits with your namespace, domain, fine. But I didn't really do it enough, so I should have done that more.

I was also looking at import statements, but I was more focused on the acceptance criteria.

I have to be honest and say that I have used Claude for most of it. For me, it is that I have plotted the task in, and then I have reviewed. Then I sit and follow what changes it has made.

Well, I would say that's what you should have done. And I think in reality...

Yes, CoPilot, where I then had access to several different models. And I'm used to using that at work, so I use it a lot for routine work. If I have to make a DTO or something like that, I can sit and do the business logic that needs to be there at the same time. And it was a bit of the same approach here. And then make sure it knows the constraints that are there and the criteria that

are there. And then review and make sure that it does it right.

That was kind of what I did. I just noticed every time it was up there to add something.

It's funny, I think I feel a bit the other way around with AI, because I think having a thought-out structure can really help AI. Because usually I actually think they have a hard time with really big code bases.

I go through it more or less line by line, so I look at all the changes that have been made, then you can say keep some changes or not keep them all.

Files\1404-m-t Treatment

Number Of References: 5

That's also this whole AI discussion, that's also really down to this. Yes, it's great, we can make some code quickly. But are you willing to pay what it costs to have fast code? Because all the arrows don't turn right. So it's just a tradeoff. As long as we all agree that it's a tradeoff. And that thing with the visualization of: No, we have something where all the arrows are fucked up. But if we agree that this is what we sell, fine.

It's also that observability that's kind of missing in relation to the whole AI dimension, where it just explodes just in advance to what it is that we get out. And I just think that you can get that feedback pretty quickly on, okay now in this sprint, how much is our technical depth in reality on this, how has it moved? And you get feedback on that pretty quickly.

You still have to remember that an AI is basically just a glorified statistician, so as soon as it doesn't follow an architecture, it can come back at you hard. And especially when we're talking about multi-agent workflow, they have to think as different instances of the same thing. So as soon as someone starts to deviate from that, the others start to say, what's going on? And then they just take the most likely choice, and then they go off the rails. So it's actually quite important that it follows its own architecture, which is also why you have these, what should I say, rule files that you set up for them, where you tell them how to do it, and I don't think I've seen a single one of them yet where architecture isn't included in those files, because it's just really important that they follow them, because errors occur, and misunderstandings arise, especially between agents who have no human interaction, not to mention testing the work afterwards, which is hell.

But what's also the issue, as I experience it anyway, is that the AI-generated code that I've seen has been enormously convoluted. So it's simply so difficult for a human to keep in their memory something that has so many layers of abstraction applied, more or less just like that, I feel, standard. That we abstract just for the sake of abstracting and in reality maybe not so... That's how I feel AI works. And that makes the code incredibly difficult to show around on a human basis.

But also for when it's big features, that you don't tie yourself up in knots. Especially with AI development, if you just use some kind of auxiliary tool to just generate something quickly, then poof.

Files\1904-m-c Control

Number Of References: 4

I've used AI for that, where I've kind of fed it the context and the README. I've made a mandate for what architecture it should follow, unfortunately AI isn't that good at following architecture, but it was kind of made aware of, uh, that something is starting to happen with

some architecture, so if that's the case, then I have to sit and read references.

Where that thing about folders, especially with AI, it can just go fast when you go the wrong way, right?

I just fed it to a machine, then I was like, go nuts, and then at some point I was just like, I hate you, you've gone nuts.

I'm sitting with the README on one side of my screen and I can see, well, but I'm not allowed to have an import that way. Then I can see that I need some functionality, like a logger class I just made. Which I'm not allowed to call. And then I talked to an AI about how to get around it. And I ended up having a lot of doubts about how in-depth you should be, how clean it should be.

Listing 1: *Treatment 0* (OBS: Pilot)

[00:00 – 00:22] r01: So, this is just a casual discussion where weâ€™re all talking togetherâ€™feel free to chime in with each other, but try not to talk over each other too much at once. Otherwise, keep it very casual. First, Iâ€™d like to ask: what do you think about the tasks you just completed? Just a general impression. Were they hard, easy, confusing? Did it make sense?

[00:24 – 00:46] t05: I think it was maybe a little hard to figure out what I was actually doing. I didnâ€™t have an overview of what the project was really about, so it was like, do this, then itâ€™s like that, okay. So in that way, it was a little hard to know, or if Iâ€™d known a bit more about what the end goal was, it might have been easier for me to understand.

[00:49 – 01:01] t02: I also think it could be a little hard to find your way around the folders sometimes. You had to spend a little time figuring out where things actually were so you could figure out where to go. It mainly described where to place the new classes, but if you needed to make changes, you had to search a bit.

[01:04 – 01:16] t04: I also noticed that you were supposed to place them where it made sense, and not where it was convenient, when you had the choice yourself. I also found that a bit difficult when you didnâ€™t really have an overview of, what exactly am I doing? Where does this actually belong?

[01:18 – 01:24] r01: I know itâ€™s already a bit out of the way. You have some specific... Like, just think about it.

[01:25 – 01:26] t04: So, in relation to this, or just in general?

[01:28 – 01:33] r01: Specifically in relation to these tasks, was there something that said that?

[01:34 – 01:52] t04: Yeah, well, the only one I got to where I had a choice was number two. But I actually think it made sense in terms of what it was supposed to be used for, which class was supposed to use it, and where it was supposed to go. So it was actually fine to figure out when you read through it. But I kind of noticed that I was thinking about it while I was doing it.

[01:52 – 02:06] t02: I also asked the co-pilot and was like, does it actually make sense to put this here in relation to what else is there, because it can kind of scan through the whole thing.

[02:06 – 02:07] r01: Yeah. What did it say?

[02:07 – 02:09] t02: It said it made sense.

[02:10 – 02:21] t04: I also think that when I ran ArchLens afterward, I could see that I hadnâ€™t created a jumble of lines afterward, but that it actually stayed pretty much... I mean, it actually stayed the same, so I was also reassured that it was probably safe to place it there. Yeah.

[02:21 – 02:24] r01: Cool. Yeah, what about the rest of you?

[02:25 – 03:44] t01: There was a point, and I donâ€™t know if I missed something, but there was a point where I was a little confused because it saidâ€™it was just stated specificallyâ€™and thatâ€™s also what I asked about, which you couldnâ€™t answer. So that log file there, itâ€™s supposed to be in that subfolder, but then thereâ€™s another file in the domain that needs to use it. I canâ€™t do both at once, so I just did whatâ€™s in the requirements, I think. Then I just ignored that readme file. And then there was also what the others were talking about regarding the overview. I was a bit lacking in that area a couple of times, like when there wasâ€™I canâ€™t remember what the object was called nowâ€™but it was some kind of sealed local class that I moved over to another script; I think it was private too, and I had to make it public because it was going to be used in another file. This probably works fine, but there were just all kinds of alarm bells going off in my head, and I was thinking, â€œBDSA instructor wouldnâ€™t think this was very good.â€ Because I didnâ€™t really have a clear understanding of how this might affect things. But I didnâ€™t have enough context to understand why that class was private. Besides, itâ€™s just good practice to keep things private if they arenâ€™t going to be used by others. But since others were going to use it, I made it public.

[03:44 – 03:47] r01: Okay, yeah. Did you check with ArchLens afterward?

[03:48 – 03:59] t01: No, I didnâ€™t use ArchLens at all during the whole... But that was also because I already knew from the first assignment that Iâ€™d broken one of those dependencies, so I was like, thatâ€™s fine.

[04:00 – 04:01] r01: What about you?

[04:01 – 04:37] t03: I thought it was really hard to keep track of where I was. And like you said, knowing what I was actually doing. It was also very hard to know where things were going, because I needed an overview. So it took me a really long time with the first assignment just to get a handle on things and stuff like that. But I feel like, when I read through the stuff, the issues youâ€™d created were really well done. It was just that I also needed to get the code so I could create the issue.

[04:37 – 04:49] t02: I actually think those issues were described well, too. They were very simple and to the point, and it was nice to have the criteria and things like that. Sometimes there are criteria, but other times it just says, â€œDo this,â€ and thatâ€™s all.

[04:49 – 05:06] t05: I got pretty confused on the third one. I think I probably should have asked, but I actually didnâ€™t end up really understanding what it was you were supposed to do, I think. Yeah, I donâ€™t know. It could be that I was... There was one thing I hadnâ€™t read properly, which I then fixed, but I was still confused afterward.

[05:08 – 05:08] r01: Understandable.

[05:08 – 05:23] t04: I was a little confused too, but it finally clicked what I was supposed to do. I was particularly stumped by that newly implemented snapshot in some function or something, but I hadnâ€™t implemented any until I finished reading the assignment, and I was like, ah, okay, thatâ€™s how you use it.

[05:26 – 05:42] r01: Cool. Well, that kind of leads into question two, which is: can you describe how you used ArchLens in your development, and when and why you checked? If you did?

[05:44 – 06:06] t04: I used it consistently every time I finished a sub-task or task. Just to verify how the architecture had changed with the changes Iâ€™d made. And then I also realized Iâ€™d created a dependency violation in task 1. But I couldnâ€™t quite figure out how to fix it, so it stayed that way. But do keep that in mind as well.

[06:06 – 06:14] t02: Yeah, I would have done that too.

[06:14 – 06:21] t05: I think I got a little caught up in passing the tests. Yeah.

[06:21 – 06:25] r01: Okay. So you didnâ€™t open ArchLens otherwise?

[06:26 – 06:34] t05: In the end, I realized it was a mess. Yeah, so I didnâ€™t have time to fix it.

[06:36 – 06:56] t02: I also prompted my co-pilot to read the README. So when I asked it about things, it was kind of implied that it had to follow that. I just checked the questionnaire, and it looked pretty good aside from that one dependency in assignment 1. But I completely forgot about it. I was also totally caught up in just getting through it.

[07:00 – 07:00] r01: What about you?

[07:00 – 07:17] t03: Yeah, well, I saw it at the beginning when you were doing that part where we had to look at the graph, and then I kind of forgot about it while I was working on the tasks. I think I did it once in Task 1, and then I completely forgot about it. I was just trying to get that logger to work.

[07:18 – 07:44] r01: Okay. I also feel like this might answer question 3 a bit, but I mean situations where you chose not to use ArchLens, and where it didnâ€™t help. And as I hear it, it was kind of like, once youâ€™d already implemented it, youâ€™d open it up and then say, â€œI donâ€™t have time to change that.â€ Or what?

[07:44 – 07:46] t05: Yeah, that was probably it.

[07:48 – 07:48] r01: Yeah.

[07:53 – 07:58] t01: It was just me who chose to say that the part in the README isnâ€™t really that important for us to do.

[08:02 – 08:05] t02: It didnâ€™t say in the README that you had to use it. Didnâ€™t it just say you could?

[08:05 – 08:20] t01: No, it just said there couldnâ€™t be any dependencies between... The domain couldnâ€™t depend on anything outside of it, but in Assignment 1 you were kind of forced to do it if you were going to follow the requirement youâ€™d been given. And so I was kind of like...

[08:21 – 08:22] t05: Thereâ€™s something in the README thatâ€™s wrong.

[08:22 – 08:31] t01: Either Iâ€™m really stupid, or this just isnâ€™t possible. Itâ€™s not possible, so I just kept going.

[08:32 – 08:40] r01: Okay. But t02, you asked Co-Pilot to use the readme, but you didnâ€™t use ArchLens. No. Okay.

[08:41 – 08:52] t02: There was a moment where I was a little scared, and I was like... Yeah. Well. Itâ€™s not that bad.

[09:01 – 09:13] r01: Would you use something like ArchLens in a real project? How do you actually understand ArchLens as it is right now?

[09:13 – 09:49] t04: Well, I used it as part of my pipeline, so I run my test, build the project, and then open ArchLens to see the difference between the main branch and what Iâ€™ve built, as a sanity check. Both to see if it complied with the current policy for dependencies, but also, for example, if Iâ€™ve changed how tightly coupled it is. So what changes does that introduce here in the architecture. So for me, it was very much a post-pipeline thing, like a final check.

[09:50 – 10:43] t05: I could imagine it being very useful if you could somehow set up some rules for what it defines... It builds the tree; you could say that if it created an arrow from here to here, then it fails your workflow, and you canâ€™t merge. Then youâ€™re guaranteed to keep your clean code or something like that. So I think thatâ€™s pretty cool. Generally, Iâ€™m probably a bit like thatâ€™if itâ€™s not the actual content of the code where youâ€™re doing something weird, then those kinds of rules should fail on workflows. Otherwise, you canâ€™t really enforce them. For example, something like unused imports. If you donâ€™t have a workflow that checks for that, itâ€™ll still end up in your code. No oneâ€™s going to sit there and check it manually.

[10:43 – 10:43] r01: No.

[10:44 – 10:45] t05: In that sense, it could be pretty smart.

[10:46 – 10:46] r01: Yeah.

[10:47 – 11:07] t02: I really like the idea of having something visual, because otherwise I wouldnâ€™t even think about it... I mean, it would be too overwhelming to even be able to... At least thatâ€™s how I feel if you just have to sit there and clean up folders and stuff like that. I didnâ€™t end up using it because I forgot about it. If I had thought about it, I actually think it could have been pretty helpful. Yeah.

[11:07 – 11:14] t03: I feel that if I had this in BDSA, it would have helped my understanding a lot.

[11:16 – 11:27] t02: If youâ€™re working on a project where itâ€™s important, at least, that the architecture is a certain way. It should be, of course. But if itâ€™s extra important, then I think it could be very good.

[11:27 – 11:29] r01: What do you mean by â€œit should be a bit like thatâ€™?

[11:29 – 11:36] t02: Itâ€™s just something you always have to think about. I definitely feel like itâ€™s something thatâ€™s been drilled into my head at SWU.

[11:36 – 11:39] t01: Youâ€™re in for a shock when you enter the workforce.

[11:39 – 11:52] t02: Thatâ€™s probably also a bit my... I havenâ€™t worked yet, so... But I havenâ€™t encountered anything else so far. There has to be some kind of thought behind it, even if it might not adhere to the different types.

[11:53 - 11:58] r01: What do you actually think it was that made you forget to use it?

[12:00 - 12:04] t03: Absorbed by the assignments. And time.

[12:04 - 12:23] t02: I think Iâ€™m just used to testing. So there was a test, and Iâ€™m used to using .NET. Thatâ€™s just what I did. Itâ€™s what I usually do. If it had errors, or it didnâ€™t build, then I was like... I just think itâ€™s because Iâ€™m not used to using it, that there was an extra step in the process.

[12:23 - 12:29] r02: Kind of like t05 says, if it had been part of the pipeline, so to speak, when you run the tests, it would have complained too. Would you have noticed it more?

[12:29 - 12:34] t02: Yeah. Or if there was something in the acceptance criteria where it was like, â€œYou canâ€™t do this.â€ Yeah. Or maybe thatâ€™s a bit too harsh.

[12:36 - 12:40] r01: What if it had just been warnings? Do you think it would have failed?

[12:41 - 12:42] t02: I had tons of warnings.

[12:45 - 12:54] t05: I mean, if warnings can be ignored, then I guess... I donâ€™t know. So either you suppress them, or it fails on a warning. Thatâ€™s kind of how I see it.

[12:54 - 13:02] t04: If youâ€™re running a pipeline remotely, no oneâ€™s actually reading the log output from itâ€™I certainly donâ€™t, unless itâ€™s failed.

[13:03 - 13:07] r01: Okay, cool. Any other comments youâ€™d like to share?

[13:11 - 13:32] t05: I think it would be cool if it could output it in a text format. It might also be that this is how I usually run things, but I donâ€™t normally have a GUI like that, so if you could just get it to print out text, then I wouldnâ€™t have to leave where Iâ€™m sitting and working. But itâ€™s also...

[13:32 - 13:34] r02: If youâ€™re also saying that youâ€™re a Vim guy, then...

[13:34 - 13:35] t05: Sorry.

[13:35 - 13:36] r02: No, no, but thatâ€™s a good perspective.

[13:36 - 13:45] r01: So would it be that when you build it, you run a command, and then it gives you some output?

[13:46 - 14:00] t05: Yes, it would be a command, and then it runs and produces an output. Kind of like the other thing, just in a terminal. I actually think that if you already have the structure, I think itâ€™s pretty easy to do without knowing anything about it.

[14:01 - 14:06] r01: That was the whole research project. What were you about to say?

[14:06 - 14:15] t04: I just remembered that I also used it at the beginning. Just to checkâ€™does it actually look like what you described in the README the first time I looked at it?

[14:16 - 14:17] r01: Did it?

[14:17 - 14:24] t04: I think so. I certainly didnâ€™t notice it. But it wasnâ€™t like I was looking for itâ€™I was just skimming through it.

[14:26 - 14:56] t05: Can you see it now... Well, I didnâ€™t get to use it, so I donâ€™t know. Can you see it like that... I only got to see the overall graph for Application, Infra, and Domain, but that dependencyâ€™If thereâ€™s one going from Domain to Infraâ€™I know thereâ€™s a dependency, but I donâ€™t know exactly where that dependency comes from. Can you click on it and see exactly where it comes from? That dependency?

[14:56 - 14:57] r01: Yes, you can.

[14:57 - 14:58] t05: Cool! More of that, then.

[14:58 - 14:59] t03: That would have been useful.

[14:59 - 15:01] t04: Nice! I hadnâ€™t noticed that either.

[15:02 - 15:05] r01: No. Alright. Well, I think...

[15:05 - 15:11] r02: Itâ€™s 29. So thatâ€™s... Right on time. Weâ€™re wrapping up here. Yeah.

Listing 2: Control 1

[00:00 - 00:06] r01: First, Iâ€™d just like to hear what you think about the tasks you just completed.

[00:09 - 00:46] c04: I think there was a huge difference in how much you had to look at the code. Sometimes you had to look through a few more files to get a better grasp of it. Then there were others where it was just return null. Okay, so just some static crap, here you go. So there was quite a big difference, and then there was also the difference between my expectations. Because it was like, you had mentioned it as if thatâ€™s how it would be if you were sitting there working normally. So if the difficulty level is too low or too unrealistic in that case, then it gets a little confusing, at least in my opinion.

[00:52 - 01:07] c02: I also think itâ€™s a question of how much youâ€™re aware of architecture to begin with, in terms of that progression. That, at least, when I did the first assignment, and then did number two, and then when I got to the third, I was like, â€œOh man, I should have solved number one differently.â€ Now that Iâ€™m looking at the system and have a better grasp of it.

[01:07 - 01:12] r01: Okay, but did you start thinking about that around assignment three or what?

[01:12 - 01:15] c02: Yeah, I actually considered going back and force-pushing over my old code.

[01:18 - 01:22] r01: Well, youâ€™re allowed to do that, but yeah, interesting.

[01:24 - 01:38] c03: Yeah, I mean, I feel like it was really easy to just go through the tasks sequentially like that, and maybe not think too much about what I was actually coding. So I still donâ€™t really have a grasp of the entire codebase, and I just completed the tasks.

[01:40 - 01:45] c01: I agree, I donâ€™t have an overview of the architecture.

[01:46 - 01:49] r01: But now youâ€™re talking a lot about architectureâ€™why are you doing that?

[01:50 - 02:44] c04: Itâ€™s because all the assignments mentioned it. Whatâ€™s the name of it all? What does that README mention: â€œHereâ€™s some architecture weâ€™re really focused onâ€™we have to follow

this clean architecture. But it doesn't seem like there was anyone who... I mean, sometimes you can add code analysis and things like that to make sure you don't break out of these things, for example. It wasn't broken down. It seems like it was just a project with multiple folders, right? So normally, if you have that setup, you'd have separate projects for each part of the architecture. So you could manage it based on project references, so you didn't mess it up. But there wasn't really any of that either. That made it a little hard to navigate between what it seemed like was mentioned in the README that you were supposed to follow.

[02:47 - 03:01] c02: But also because the tasks kind of increased in architectural complexity, it seems like, at least the last one—the fifth one there—Today, there you definitely need more and more of an overview of the system to be able to achieve what you want here.

[03:02 - 03:14] r01: Do you feel that the README was representative of a typical project where a README is written? If you've worked on projects at work or whatever?

[03:14 - 03:15] c02: I think that's very individual.

[03:15 - 03:32] c04: I think there are some projects where they don't write a README. And then there are some where they stick to the classic open source approach, where it's like, "Here's how you run the project." You need these things to run it. And then there are others where they say, "These are our coding standards for this project."

[03:36 - 03:44] c03: At my old job, for example, there was nothing like that at all. With the architecture, it was just a matter of figuring it out.

[03:44 - 03:45] c04: That kind of thing—we ship code.

[03:48 - 04:07] c02: I also think it's that tricky distinction between what represents real life, where you go out and code something you don't know what it is. Or do you need a more thorough introduction to what the system actually is and what it's supposed to achieve than what's necessarily just in a README. There's also a context around development that can be a bit tricky.

[04:08 - 04:16] r01: Okay, so how did you approach this architecture or dependency policy? How did you tackle it?

[04:18 - 04:27] c04: I looked at it, and then I saw everything was named Infra, and I was like, ah okay, so I have to use everything from Infra. That's as far as I got in a check, right?

[04:29 - 04:35] c02: I think I broke it pretty quickly, too. That's why I went back and was like, ah shit.

[04:38 - 04:45] r01: But did you guys just have it in the back of your minds, like an idea of what was in the README?

[04:47 - 04:48] c04: I checked it a couple of times.

[04:48 - 04:58] c01: I think I forgot all about it when I started coding.

[04:59 - 05:11] c03: I tried to stick to that onion architecture thing because I'd heard about it in BDSA. So I had it in the back of my mind, but not because I used the README.

[05:11 - 05:38] c02: I got a little stuck at the end because I had two things on my mind: what the program does, and then how it's abstracted and structured, and I couldn't really map those two things in my head because I don't know the system. There's a config somewhere, but where does it fit in the layers? The whole process of establishing the domain probably takes longer, than when you have to complete some tasks in an hour and a half.

[05:38 - 05:46] r01: Yeah. What do you think would have made it easier to adhere to the architecture?

[05:46 - 06:06] c01: Well, I think that if I'd had a slightly more thorough... where I'd looked around a bit more at what the architecture is, where things are, and what's called what. Yeah, getting that kind of bird's-eye view could have been helpful, I think. I think very visually.

[06:08 - 06:28] c04: It could have already been... There was already an idea of which folders existed and stuff like that. You could have just said, what's here goes in this layer, and this goes in that layer, or whatever. Then you'd just have a quick table that simply said, this is how we've structured it. Now that there aren't really project references to handle that instead.

[06:29 - 06:39] c02: Sometimes, if I have to go in and code on a system, or get to know a system, I actually sit down and draw it out, sketch out roughly how the classes are, because you're mapping out your own domain model.

[06:41 - 06:57] c01: I looked at the diagrams in the README, but they didn't speak much to me, because there wasn't much... Or well, maybe I didn't read it properly, but if I had read it a little more carefully, I might have a slightly better overview of the architecture.

[07:00 - 07:05] r01: Is architecture something you usually think a lot about when you start a project?

[07:07 - 07:38] c01: For me, it's something I think about when I realize that this isn't working very well. And then you think, what kind of architecture could improve this. And then you start moving things around and stuff like that. But it's kind of something I usually think about afterward. While there are perhaps many who do it the other way around. They start by thinking, "This is the architecture we're using," but I think I tend to find the architecture that makes sense based on what I've already built.

[07:40 - 08:13] c04: I completely agree. I also think that if you just try to follow a predefined architecture, it's like trying to force something into a template, but often the right approach is to use what you have and then build a clean pattern with these security considerations in mind. We know that this shouldn't be called, or this needs to be wrapped around that, so we can secure these things. And all those who split their code based on what you're doing, because it's always different.

[08:13 - 08:30] c02: There's also something to the reality you're mapping in your system. In some systems, it makes sense to have certain things; in others— but even just the domain you're looking at can determine whether there are tasks that need to go through queues, which also defines how you

think about it.

[08:30 – 08:42] c01: There's also the idea that the architecture represents the company.

[08:42 – 08:43] r01: Conway's Law?

[08:44 – 09:07] c01: Yes, that's behind it. So when I was just sitting alone, I don't think that when I'm programming by myself, I often forget... to do it properly after all, it's just me looking at it. But when you're part of a larger team, I quickly realize that it makes perfect sense to have an architecture that makes sense, because otherwise people just get so confused by the amount of code.

[09:09 – 09:20] c04: Then it quickly becomes: Why is this being used here? What does it even do if it's here this is peeling bananas, why do I have a washing machine over here?

[09:26 – 09:37] r01: You mentioned that people usually organize things into projects instead of just having them in folders. Do you think that usually works best?

[09:37 – 10:07] c04: It seems to work. I mean, it depends a bit on the size of the project, but it's also something where, if you really want those strict boundaries between things, that's usually the best way, I think. Otherwise, I think you need some kind of code analyzer that checks where namespaces are being used. It's kind of an either-or situation, because if you want to enforce something, you have to enforce it. Otherwise it just becomes a game we're playing, right?

[10:08 – 10:16] c02: Yeah, and when is the architecture mature enough that you dare to enforce something? When you're not still in the process of domain-mapping something?

[10:18 – 10:38] c03: I just want to say that I agree too, because discipline is really important when you're working on a codebase. If you don't have that separation where the technology fails if you don't do it properly then people are like, "Yeah, we're a bit busy, let's see if we can get around it." If you want to be fast, you just break those rules.

[10:39 – 10:43] r01: Yeah, have you ever worked on a project where that's enforced?

[10:45 – 11:26] c01: Yeah, it gave me a bit of a headache once. It was kind of that thing you were talking about, where people tried to say, "This is how we do it because that's what people say is the right way to do it," but it just didn't quite fit the project. There were too many of those different areas. There's kind of a sweet spot, isn't there, between dividing it up too much or too little. It can sometimes be a bit hard to hit that spot. Especially if you make the decision before you start the project. You might not even have an overview of how it should be divided up in the first place?

[11:28 – 11:37] r01: So what would be a good approach? What do you think would be the best way to get around that situation in a scenario like that?

[11:37 – 12:06] c01: Yeah, it's difficult, isn't it, because you can't just... I mean, there are these principles called AHA's to avoid hasty abstractions where you first just try to get it to work, and then afterward it starts to click what the good abstraction or good architecture is based on this. So generally, try to postpone a bit, to start enforcing a lot of things before it makes sense. That's what I generally believe.

[12:08 – 12:24] c02: I suppose it's also a question of how much effort you put into doing that in which phases of a project. If you're sitting there developing an insanely in-depth design document on how the architecture should be based on what classes should look like. Or, as you say, okay, Wild West we'll figure it out.

[12:24 – 12:28] c01: We need to build a prototype; we need to get started, and then we'll figure it out later.

[12:28 – 13:26] c04: It's usually a good mix of both. A good project always gets off the ground if you know what you're building. So if you already have an existing project, you can build it based on that. Or if you just have an idea of what it is, then you'll have to build the idea first. So you build an MVP. Once you have the MVP, you can just say, okay, we've learned these things about our domain. Okay, so we can map it out in some documents and say, "This is how it should be," and then you can build it based on that, right? Because otherwise, you end up in that situation where it's just a game you're playing all the time we're just business people who all sit there writing Word or Markdown documents all the time, and get some idea that, "Ah yes, we're so smart because we drew on a whiteboard five minutes ago, instead of actually knowing how things work."

[13:26 – 13:46] c02: I've also been on a project where you sometimes have to fail fast, right? Because otherwise I'm sitting there writing an insanely well-documented design document 100 pages on exactly how the architecture should be and then you spend a month coding, and suddenly you're like, "This actually doesn't make sense in relation to what we're trying to achieve." Okay, we have to go back and write a whole new document.

[13:46 – 13:52] c01: That's it, it's like a false assessment, right? Versus having a slightly more iterative approach to architecture.

[13:54 – 14:01] r01: Okay. I'm actually also a bit curious how important do you guys actually think it is to have architecture?

[14:03 – 14:49] c04: Again, it depends on the scale of the project. I mean, how many people are involved in producing it. If you're one person and you're building it for 10 people in your family or something like that, then it's probably not the most important thing. If it's a security-related project and you're deploying it to a lot of people, then it almost doesn't matter how many people are coding it. In that case, it might be very wise to put security barriers in place within your infrastructure. And if there are many people coding on something, it also becomes really, really important very quickly. But again, you need to strike that balance between when to introduce more of these architectural barriers into your system, right?

[14:52 – 15:08] c02: But you can also abstract a domain to the point where it's completely unintelligible

. Where youâ€™ve pushed the abstraction layer so far that you donâ€™t really know what the program does . Or people coming in from the outside as new contributors canâ€™t really figure it out because itâ€™s too complex.

[15:08 – 15:24] c01: I think the most important thing is to keep going back to it: are we adhering to it, does it make sense, or is it just becoming frustrating to use? How can we make it better, so that we can continuously adapt it to the project.

[15:25 – 15:29] r01: Really annoying question, but how often do you think it should be evaluated?

[15:31 – 15:42] c01: Yeah, definitely more than a couple of times a year, but a little less than a couple of times a week.

[15:44 – 16:44] c04: It depends a bit on how much churn you have. If you go and push a new major version every month, then it may well be that if you push that many major versions, you need to consider whether you need... Major versions often mean that youâ€™ve made a structural change that then affects those who use your system. You might want to consider whether you have the right infrastructure setups in your system if you keep changing everything all the time. But if you have a system where you never, ever need to change anything, you can never find a reason to release a new major version. In that case, you might be sitting on a solid infrastructure with your environment. But you shouldnâ€™t change something that works. Try to think about how many major changes we keep having to make. And can we fit this into a few more boxes, or whatever makes sense?

[16:50 – 16:53] r01: Do you have any other comments youâ€™d like to share?

[16:55 – 17:09] c03: I think that if Iâ€™m actually working on a .NET project, you might already start thinking, okay, so I need to think about architecture. Because itâ€™s such a big tool, there are so many things. I mean, if youâ€™re working on a small Python project, then I get it, okay.

[17:09 – 17:30] c01: As I also wrote in the survey, when Iâ€™ve generally worked on .NET projects, architecture has really played a major role. Whereas for other projects, itâ€™s been a bit more loose. So you figure it out. Itâ€™s also just so built into the framework. So in that way, it handles dependency injection.

[17:30 – 17:42] r01: Mmm. Yeah. Cool. Thanks.

Listing 3: *Treatment 1*

[00:24 – 00:29] r01: Well, let’s jump right into it. What do you think about the tasks you just did?

[00:31 – 00:38] t07: I think 1B was really hard. I don’t know, I can’t remember the wording now.

[00:38 – 00:39] r01: What made it hard?

[00:39 – 01:09] t07: I just think I didn’t understand the wording. And then like, yeah, serialize. There were a lot of files. I didn’t know where to go. And then I think I prompted my Claude. Not Code, but just the regular one. And it totally drove me crazy for an hour. And then I realized it was completely crazy when it asked me to change your test, and then I thought, let’s revert back and find a better way, and then it went better.

[01:12 – 01:15] r01: So it was about getting an overview of the code or what?

[01:15 – 01:18] t07: I think it was just understanding the task. That was the problem.

[01:19 – 01:21] r01: Okay, and Claude didn’t understand it either?

[01:21 – 01:27] t07: No, Claude didn’t understand it either. The first time. But it did after it was just whipped a little more.

[01:30 – 01:41] t09: I would say that the tasks were very clearly defined. Because I straight up just copy paste into Claude. Hey Claude, I’ve got this task. Go. And then it solved it for me.

[01:43 – 01:44] t10: What?

[01:44 – 01:59] t09: And then it just did it. So in that way they were very much like... The thing about there being acceptance criteria. That was good. That’s good for an AI. Because then it’s like, check, check, and your tests also fit, check.

[02:02 – 02:03] t06: I had the same experience.

[02:03 – 02:09] t10: I have a plus one on t07, that is. That GitHub Copilot there.

[02:09 – 02:11] t07: The others were just fine. Only 1B was...

[02:13 – 02:30] t10: It was a bit long compared to the others. I could have used a little more time sitting down and looking at the code before I started the task. And understanding the context. Yes, maybe I’m a little too inclined to just trust some AI that you haven’t spent enough money on.

[02:36 – 02:46] r01: Can you explain a little bit about how ArchLens was part of your development? I mean, something like when you opened it up and why?

[02:48 – 03:08] t08: Mine was a little delayed because I had to push it first, and then I had to check it from another computer. But I used it precisely because I was in doubt about, okay, we have these models, we had a domain and infra and app. And then I started using that logger, and then I could see that there are some things here that are starting to cause problems. And then I used it to just double-check.

[03:08 – 03:10] r01: What do you mean it started to cause problems?

[03:10 – 03:42] t08: So I made a... you know you had to make the logger in Infra, and then you had to use that deserialize thing. And then it became like a dependency on Infra, and I’m not allowed to be, so I used it to double-check. Okay, there’s a dependency now that’s going the wrong way, so I just moved that deserializer out of the domain, for example. Then there were no errors in it. So I used it as a kind of result to see, okay it still looks right, instead of having to sit around and hunt those down myself.

[03:42 – 03:44] r01: But you had a feeling about it before?

[03:44 – 03:57] t08: Yeah, I just used it to double-check. It’s easier than having to sit around and double-check that it’s actually there. But that’s because I haven’t done much clean architecture, so I had to think about which way they were pointing.

[03:57 - 04:12] t10: Yes, I agree. I also used it all the time when I wrote some code or did something new in a different class than the one I had just worked in, to see, okay, did I make some dependency that I hadn't thought of, or was there something? Yes, there was a couple of times.

[04:12 - 04:14] r01: Did you have it open all the time?

[04:14 - 04:31] t10: Yes, it's a bit like that, because my VS Code wasn't that big, so I had a lot of windows, so I kept closing it and then opening it again. And then sometimes I would compare it to the base branch to just see if there was supposed to be one here.

[04:32 - 05:07] t06: I did that too now, after each task I went to the other PC, pulled down the branch and then I just looked, looked a lot at this remote branch diff so you could see what was new in that commit where there was a new dependency and see if it works right and then, yes, that's what I used it for anyway. That rough UI I would like to have, you could... I think at least hot reload or something. So while you're sitting and developing, I think I used it more.

[05:07 - 05:09] r01: Yes. What about you?

[05:09 - 05:40] t09: It's the same. Every time you had solved a task, now I've also used Claude to solve them, so it was a little more to keep an eye on, okay, are the dependencies so... Which ones have you added? Added the ones that I expected it to add? And did it violate the dependency policy? I only discovered the dependency policy a little later, so I had to go back and be like, did I violate it earlier?

[05:43 - 05:46] t07: I don't think I discovered a dependency policy, I would say. It sounds a little strange. I think a lot of lines came up, but I didn't really think about it. Maybe they shouldn't point to those things now.

[06:00 - 06:01] t09: I also discovered it a little too late.

[06:01 - 06:09] r01: Yes, because now we're talking a lot about architecture. What prompts you to think so much about it?

[06:09 - 06:27] t06: Read the specs. Just make it read the README, make it read the how-to-make. And then have them in the context all the time. And then give it like tasks. After a task is compacted, so you still remember those two pre-injected MDs there.

[06:27 - 06:30] r01: Okay, so you gave it?

[06:30 - 06:48] t06: I compacted it between the tasks all the time, so it became task specific. But so I didn't have to fill it in every single time with those two MDs there. And then say, we're going to make the new two. And then it was written up in a nice way. So it was very spec.

[06:50 - 06:51] t09: It was smart to give it.

[06:52 - 06:54] r01: Do you want to do it normally, so give it...

[06:58 - 07:04] t06: I also made a Claude init file, so that it kind of...

[07:04 - 07:04] t08: Hahaha, so it was all Claude

[07:06 - 07:56] t06: I started by saying, read these two, and then explain to it what we're going to do now. We're going to do these tasks. I made an init file based on that, and I just started feeding it sequentially, what's it called, the tasks. And then change branches. So I didn't really make a real branching strategy, but... What can you do. I don't even know those git commands. I also have it on my Claude, I can show it here, because it's a lot of context management, so on my Claude, now I just have to show it here. There I have something like blah blah blah. There I can see when it fills up the context window down there, I've kept it below 50% the whole time And then I compacted it there. So it wouldn't be silly. So it wouldn't be silly. Allegedly, allegedly

[07:56 - 08:00] r01: Why did you guys bother to adhere to the architecture?

[08:04 - 08:05] t08: That's what it says there, we see.

[08:06 - 08:09] r01: Yes, in the README.

[08:09 - 08:11] t06: So the branching strategy, or what are we talking about?

[08:11 - 08:20] r01: Yes, we're talking about the architecture. Because you talk a lot about adhering to the architecture, but...

[08:20 - 08:31] t08: You've described it, so I thought, so... And it said policy right, so I said... If it wasn't in there, I wouldn't have thought about it, I guess. So I would have just... Driven off and just made it work, and not really thought about...

[08:31 - 08:31] r01: Yes.

[08:32 - 08:32] t08: Yes.

[08:33 - 08:33] r01: Okay.

[08:33 - 08:38] t08: But there were also pictures and stuff, okay, and this one only has to know about this one, and this one only has to know about this one, then you're kind of okay.

[08:39 - 08:39] r01: Okay.

[08:39 - 08:44] t08: Now I probably shouldn't really destroy it. Even if no one notices it.

[08:45 - 09:01] t09: Yes, you would do that in reality, if you were to contribute to some open source code, it won't be merged if you don't follow the rules that they've set up. Yes.

[09:01 - 09:18] r01: Okay. Yes. Cool. Were there any times when you chose not to use ArchLens? Or when you didn't find it helpful? What were those situations?

[09:18 - 09:46] t06: Mid-development it didn't really make sense to me. I don't think I would have used it either, even though I had it on the PC. I don't think I would... Now I say I wanted hot reload, but I feel like it's a bit more like you can sit back and look at, okay, I mean, I maintain these architectures, what's it called, principles. And otherwise you can go in and look more, but I don't think I would look at it while I'm sitting there developing.

[09:47 - 09:47] r01: No.

[09:47 - 09:49] t06: I think it's more of a review for me.

[09:51 - 09:56] t09: Agreed. Also, I didn't use it to understand the codebase at all.

[10:02 - 10:10] t07: Yes, I would also say plus one on that one. It was also always when I finished a task. Now I just have to remember to go in and look here. Look where I had made new arrows.

[10:10 - 10:19] r01: But when you discovered the new arrows. Did you revisit it, or were you like... I have

one more task, I have to get to it?

[10:20 – 10:30] t09: If there was an arrow that wasn't supposed to be there. Then you went back and fixed it. Or at least that's what I did... It wasn't that good, move on.

[10:30 – 10:30] t08: I was so busy.

[10:30 – 10:31] t07: That's the strategy I've taken.

[10:32 – 10:38] SPEAKERS: (Muffled talk)

[10:38 – 10:45] t10: When you saw that arrow that wasn't exactly between the two right ones. I was like, okay shit, now I've done something.

[10:47 – 10:53] r01: How did you feel about it not being forced, that you're not allowed to do that arrow for example?

[10:53 – 10:54] t09: What do you mean?

[10:54 – 10:59] r01: So that it might have... made a compile error

[10:59 – 11:03] t09: But I think maybe I could have done it better, because my code might have been a little cleaner. If it had just said no, you're not allowed to do this.

[11:04 – 11:08] t08: I think it would be great if it was part of the suite. I can see that you're saying to a pull request, you can't do that

[11:10 – 11:13] t10: It would be great if it was an automatic check when you write the code.

[11:13 – 11:16] t07: Yes, a linter or something like that

[11:16 – 11:17] t09: Definitely

[11:21 – 11:30] t06: I'm just sitting there thinking fuck, that would be annoying, so when you're sitting there and you have to push something, and then those arrows come up. It's like... Fuck

[11:30 – 11:39] t09: Either it does it, or your senior does it. Better that someone protects you before someone shames you.

[11:39 – 11:39] t06: Really.

[11:41 – 11:52] r01: Um, did it also change your way of thinking about the Dependency policy, or did it mainly just confirm something you already knew.

[11:52 – 11:52] t06: ArchLens, then.

[11:53 – 11:55] r01: Yes, using ArchLens.

[11:55 – 12:10] t08: That confirmed it for me. I knew what it was about before, but it's hard to visualize it sometimes. Then you kind of have to look through some files and check oh fuck did I call application inside the domain somewhere. Then it's easier to just see an arrow.

[12:12 – 12:23] t09: 100%. Also in a time when AI is being used more and more to code, it's nice to say bro, you've fucked up.

[12:23 – 12:37] r01: But it's also, as you mentioned earlier, that you used Claude, and then you checked Claude's work with ArchLens. Did you feel like you looked at the code the most, or did you feel like you looked at ArchLens the most?

[12:39 – 12:56] t09: I looked at the code to check the acceptance criteria, and then I used ArchLens to check the dependency policy. It looked because it's impossible to sit, okay I'm in this file, and I've imported this file. It helps a lot, just having a graph, like there's a wrong arrow there, I can see.

[13:00 – 13:02] r01: And you didn't just think of import statements, did you?

[13:03 – 13:13] t09: Well, I didn't look at import statements, but it's very nice to have a check like that to make sure Claude didn't hallucinate that it was supposed to import something from something it wasn't supposed to, right?

[13:13 – 13:22] r01: Makes sense. Yes. What about the rest of you? Was it also just confirmation of something you already knew?

[13:22 – 13:42] t10: Yes, I think so. But it was nice to have. Because often, when you're sitting by yourself, you have the idea of the architecture in your head. But now it can say in black and white that you've made a mistake. It's easier than having to sit and keep track of it yourself. Or having to sit and read through a lot of files and draw a diagram yourself. What's cool is that it can just tell you in half a second that you've done something stupid.

[13:42 – 13:53] t07: And click through those different folders. And God, where was that file, it was here. Instantly. That's a little cooler. But hot reload could be cool.

[13:53 – 13:53] t06: Hot reload was the coolest.

[13:53 – 13:55] t07: Instant.

[13:56 – 14:06] t06: But more as a part of some CI/CD workflow when you push, than just something for a PR, it spits it out for a preview or something, I don't know. So you can sit and watch.

[14:08 – 14:14] r01: Do you think it's important to adhere to the architecture of projects?

[14:15 – 14:17] SPEAKERS: Yes... (muffled agreement)

[14:19 – 14:21] r01: What makes you say that?

[14:23 – 14:40] t08: So if you use these templates, these architectures that you know, it's much easier for new developers who come in, and then you know how it works in some way. And then you can also make sure that you don't do some completely crazy things when you have these rules that we follow in the company, and everyone knows the same thing.

[14:41 – 15:13] t07: The place where I work right now, you can't be guaranteed that a controller is a controller at all. Things like that, so it just makes it really complicated to find your way around the code base And you really have to, you really shouldn't trust that just because it has controller in the name that that's what it does. So that makes it a little more complicated, I would say. So it would have been nice if they had stuck to the rules that are in place. Yeah.

[15:14 – 15:18] t06: Not naming any names here. There are no names. They're in the middle of merging. No.

[15:20 – 15:22] t08: Not on tape here.

[15:24 – 15:28] r01: No. Is this something you'd use in a real project?

[15:30 – 15:31] t06: Then it should be part of a workflow like that.

[15:32 – 15:38] t07: If the UI was a little nicer to look at. Because it looks a little rough.

[15:39 – 15:39] t09: It looks like a school project.

[15:42 – 15:43] t08: It looks like 25 people have downloaded it.
 [15:47 – 15:50] t09: 26, 27, 28, 29, 30.
 [15:53 – 16:03] t06: It could also be useful if... but then again AI, that has also been a revolution. But if you could create an abstraction layer that was also more readable for AI, that could kind of... Don't have to read all sorts of files, and all sorts of things that clutter it, if you could just get the pure graph representation spit out and then match it up against a spec or an architecture, where you have to have a convention with a name or something.
 [16:21 – 16:28] t07: That would be fucking smart. Then you could just check before you do your PR. Look, check out my...
 [16:28 – 16:29] t09: Because it has to create some kind of graph structure behind it.
 [16:29 – 16:33] t06: Yes, but it just reads so much.
 [16:33 – 16:42] t09: But if you can spit out both the UI, but also the raw data.
 [16:42 – 16:43] t07: Yes, true.
 [16:43 – 16:44] t08: Just a binary.
 [16:43 – 16:48] t06: Just a skill. A small skill.
 [16:48 – 16:49] t07: Check the architecture.
 [16:50 – 17:04] t06: I haven't been on a project for two months now, so I've just been sitting on Hacker News and reading all the different flavors of the AI.
 [17:04 – 17:27] SPEAKERS: (Loose talk unrelated to project)
 [17:27 – 17:30] r01: I was going to say, are there any other comments by the way?
 [17:32 – 17:33] t08: No, it wasn't anything productive.
 [17:33 – 17:34] r01: It wasn't any?
 [17:34 – 17:36] t06: It was nice.
 [17:36 – 17:37] t10: It was nice.
 [17:37 – 17:38] t09: It was so nice.

Listing 4: *Control 2*

[00:17 – 00:31] r01: So now we'll just do a little summary, debrief together. So just to hear what you think about the tasks you've done overall? If you want to put some introductory words on it.
 [00:31 – 00:55] c05: I managed the first two, I think. Normally, when I'm so bad at a language, for example C sharp, I would constantly ask my colleagues about things. It was a bit strange not being able to do that. But I think it went okay, after I just remembered that it was object-oriented.
 [00:59 – 01:41] c07: I think it was a very good... Good challenges in terms of testing whether we could keep track of the architecture that was set. It's kind of funny when you're sitting around and used to constantly talking and bouncing ideas around, and it went really well at first, and then at some point my brain took over and was like, okay, we're going to get it done. It was like a consultant's brain, that's how we're going to get it all done, and then it just went faster and faster. The quality maybe dropped a little, but good challenges.
 [01:43 – 02:01] c06: I can understand that consultant's brain, where it's like it has to be completed. Then the quality doesn't matter. I've probably screwed up the architecture a bit now and then. So it's a nice challenge.
 [02:01 – 02:05] r01: Can I ask what makes you mention the architect?
 [02:07 – 02:16] c06: The questions, probably for me, afterwards. It just made me reflect back on what we were actually doing. So that was why for me.
 [02:17 – 02:20] r01: The questions... can I get you to be very specific.
 [02:20 – 02:37] c06: There was one of the questions that asked about something like, have you complied with this architecture stuff, the picture you had, etc. And that's what made me reflect back on it. Was it specific?
 [02:37 – 03:07] c07: I think very specific for me with that, it was as soon as in task 1, we're going to make this logger as part of our infrastructure. And as soon as we get to task 1c, we're going to use this logger very specifically in this application. And then I was like, where is it called? Was my first contribution. Because I noticed... That's when I realized that okay, and we're going to have to do dependency injection, something needs to happen here. And so from that task forward, I was very focused on that.
 [03:08 – 03:09] r01: Okay, yes.
 [03:09 – 03:58] c08: Well, I think you've described in the README, you've described something, plus the images that aren't there, some kind of clean architecture. You had kind of set some constraints on the domain and application and infrastructure. So it was like, based on that, there's some kind of architecture and the coupling was also, it sounds like clean. You probably had that in your picture too, and it's like, again I think I was like, okay we have an hour and a half, or we have some timeframe, so like, I don't pay too much attention to the details, but that, it sounded like a hard requirement, and then yeah, that dependency serialize builder graph, it was kind of clear to say, where are you, you're going to have to do some dependency injection to comply with this requirement. So I had to make an interface... Well, there's probably a lot of ways to do it.
 [04:06 – 04:10] c05: But it sounds like you looked at what the graph put out. Or what?
 [04:10 – 04:11] c08: No.
 [04:11 – 04:12] c05: Okay, fine.
 [04:13 – 04:43] c08: But it was... No. One of my comments, what I wanted to say, before the record, was because I actually think it could be interesting if you could see this graph being used live, so you could spot this Architectural Erosion or something, what's happening right now, now I change this or some crap and then all of a sudden, then I can see, okay warning, or also if I could just see the graph live, then I could see, well but now there's a break in that, which in my own little head I had a model of, but that mustn't happen.

[04:45 - 04:50] r01: The graph that was generated, it wasn't from the repo, it was from test?

[04:50 - 05:27] c08: Yes, exactly, that's what I mean. I think it could be exciting while you're coding live, then you can see an update of the graph. If you have a Plant UML render server like this running, you could spin it up like this on change or whatever it was. Then you see your changes. Of course it's difficult how quickly you can bake some of the things in. But if you have it visually, and it's also something with the scale of the repository you're looking at, because now it's very small. How does it scale? That's of course something else. There are a lot of things you have to think about. How do you constrain and look at it? Does it only look at this part of the architecture? Otherwise it becomes a very large Plant UML diagram very quickly.

[05:28 - 05:44] r01: Can I ask how you approached this dependency policy? Did you have it in mind when you were coding or something? How did you use it?

[05:45 - 05:48] c06: I probably forgot about it a bit. I want to be honest, I forgot about it. So, yes.

[05:48 - 06:01] c08: I have to be honest and say that I have used Claude for most of it. For me, it is that I have plotted the bag, and then I have reviewed. Then I sit and follow what changes it has made.

[06:01 - 06:04] r01: Did you give it the dependency policy?

[06:05 - 06:57] c08: I gave it the README and how-to-complete-tasks, and if I may elaborate, I asked it to extract, what do you understand from what you've got now, what constraints are there, what rules are there, and then it spits it out, and I think, okay, it's got it. And then I sit and look at the other side and see if it looks about right. And then I just look for some patterns. Has it done dependency injection here? Has it made sure to extract those things into a class, as it should be? Okay, has it done a getter here? Does it follow the criteria? Does it follow the acceptance criteria? Yes, it's not because... I have an interest of time, so I don't have 100%...

[06:59 - 07:03] r01: Would you do that for a normal job? So give Claude the README for example?

[07:03 - 07:40] c08: Yes yes, without a doubt. If I may, I mean the consulting world here, not all clients think you can use AI, it can be proprietary software. So there are a lot of things, it depends on where you are in the development phase about what you can do. What do you have access to? So everything that is kind of non-disclosure, I can't cram in. You can also have local agents, so if you run local LLMs or something, then you can do that, so you can do some things, but there can also just be a hard demand from someone that, hey, you can't use agents. And then I wouldn't be so quick. Then I would need a little more time to get the task done.

[07:40 - 08:03] c05: I mean, for my sake, I only got to the first two, and I was kind of paying attention to the first one. Also, I spent so long figuring out how the hell I do this, but I also checked that you've done this with Dependency Injection. That was by far the thing I spent the most time on, but I used it again too. I'm pretty new to using AI to program, but I think I managed it.

[08:05 - 08:12] c07: I think I've had a very similar workflow. I use the agent in the editor, so it has access.

[08:12 - 08:13] r01: Excuse me what? Well, Claude? Claude Code?

[08:13 - 08:43] c07: Yes, CoPilot, where I then had access to several different models. And I'm used to using that at work, so I use it a lot for routine work. If I have to make a DTO or something like that, I can sit and do the business logic that needs to be there at the same time. And it was a bit of the same approach here. And then make sure it knows the constraints that are there and the criteria that are there. And then review and make sure that it does it right.

[08:43 - 08:47] r01: So you also gave it the README? And also how to deliver tasks?

[08:47 - 09:10] c07: Yes, it had access to everything in the code base. When you run CoPilot, it has access to run tests, and then it can see that there were tests that failed, and then it can say that I think that's why, so it can try to fix it. It can really do a lot while you're sitting there coding. So that way there is a good interaction with your own agent in some way, who has a good overview.

[09:11 - 09:33] r01: But how, so did you check the code... Well, I hear a bit that you checked that it kind of lives up to the architecture or dependency policy. How, if you... did you do that, and how did you do it?

[09:35 - 09:46] c07: I go through it more or less line by line, so I look at all the changes that have been made, then you can say keep some changes or not keep them all.

[09:46 - 09:52] r01: I'm thinking specifically about the architecture or dependency policy.

[09:52 - 09:59] c07: I primarily kept an eye on it so that it doesn't start importing and using things across the layers in a way that it shouldn't.

[10:00 - 10:06] c05: That's kind of what I did. I just noticed every time it was up there to add something.

[10:07 - 10:12] r01: Yeah, okay. So you were reading import statements a lot manually?

[10:13 - 10:19] c08: Well, I would say that's what you should have done. And I think in reality...

[10:19 - 10:20] r01: You relied on it?

[10:20 - 11:37] c08: That's kind of funny. I was also looking at import statements, but I was more focused on the acceptance criteria. I was checking a lot more because it mapped the logic over. Now it has moved something over and refactored something in another class. Then I sat and tried to keep track, does this now match? Have you done that method correctly? But I actually had more of a flow where I side-by-side, so I just made it do a draft pull request, and then tried to look at the changes, and the acceptance criteria, and compare what the code was. Also because some of the tasks, it wasn't because there was so much change, that it was really just refactoring something, and precisely, well, that could also be me completely wrong about it. But to me it seemed like it wasn't because you had to write so many lines of code necessarily. Well, it was more about getting an understanding and an overview. But in hindsight, I should have looked a lot more at import statements. So I looked at that to start with, and I was like, yes okay, you've made a new interface. Boom, it fits with your namespace, domain, fine. But I didn't really do it enough, so I should have done that more.

[11:37 - 11:54] r01: But do you think that if it wasn't so clearly stated in the assignment or the README, would it be something you would have paid attention to? Is it something you would have paid attention to in your own jobs?

[11:54 - 11:57] c06: In relation to the architecture?

[11:57 - 11:58] r01: Yes, or in relation to import statements and dependency policy.

[12:04 - 13:00] c07: Precisely with what I'm doing, it's important because we work across. We always work with a C-sharp project, and across that there are quite a few different projects inside it. So it's quite important that you don't start referring back and forth with different circular dependencies, because it just creates an incredible amount of mess with many of the packages we're creating or using. Also because they are proprietary in some form or another. There are some packages that are developed in-house that we use for development. It is also not something that we want to sit around and have as a dependency in all our projects. So it is very important that the structure is complete to a certain extent. And then you can move around if you need to change something. So I always keep that in mind when I code, and try to keep track of the right things in the right places and handlers and models etc. There is such a very strong structure that we follow.

[13:01 - 13:35] c05: I would say, out at my job, because I sit and work with mainframe code, most of the code I work with is older than the Clean Code book. And much older. There is no real pattern in how to do includes and things. And therefore, if you tried now, well, it is a gigantic code base. It would be uphill. So of course you try to do something that's not too crazy, but sometimes people do.

[13:37 - 14:16] c06: I think for me so often I've noticed that I make sure to really adhere to the architecture if... It really depends on, for me, the codebase. The bigger the codebase, the more I want to adhere to it. How much smaller, it's quick enough to refactor it from now on, because there's only 3 packages or something like that. 3 dotnet packages, it's easy enough, I don't have to think about it that much. But when I suddenly looked... at my previous company with these big codebases for some big customer, I got really into it, because I knew that it could break a lot of things all of a sudden. And I couldn't just refactor it.

[14:16 - 14:18] r01: Do you think it's important to adhere to it?

[14:20 - 14:39] c06: I think again, for me it's a bit back to the size. Do I know it's something that's going to be big, or something that is big? So for me, yes, it's important. If it's something small, today at least, especially with AI, then it's like, no, not really, it can be fixed quickly. That's also maybe a naive way of approaching it.

[14:41 - 14:50] c05: It's funny, I think I feel a bit the other way around with AI, because I think having a thought-out structure can really help AI. Because usually I actually think they have a hard time with really big code bases.

[14:51 - 15:19] c06: That's also the thing with large code bases, so if you know it's going to be something that's going to be big, that's where I get into the architecture at some level, because I know it's going to be complex, but if it's just my own hobby projects or something that I'm developing for Campus Cup, then it just has to work. The architecture is less relevant as long as it's still fairly readable afterwards.

[15:19 - 15:50] c07: The readability thing is pretty important. In our department, we're both... I sit and make projects and then when they're sent out into the world, people in our own department sit in a group of people who kind of maintain and answer tickets and support the systems we've built. So when you sit and look them in the eyes every day, and have lunch with them, then you are like, it is also important that the projects I launch are something that they can be allowed to fix in a proper way. So the structure should preferably be there to some extent, even if we don't have the strongest rules. When I was hired, no one said that this, this should be here.

[15:50 - 15:56] r01: Are you thinking in terms of onboarding or communication to non-technical people?

[15:58 - 16:33] c07: I was thinking primarily about onboarding and then continuing projects when someone is going to take over. We have had a lot of problems with some people having finished half a project and then they stop. And then a new consultant has to come on, and then it's like, what has been done here? So the better you can do it from the start, the easier it will be for others like that. And when you yourself are on the receiving end of such bad projects, you want to try to do better going forward. There were certainly some challenges there.

[16:34 - 16:40] r01: What do you think could have made it easier to follow the architecture or these dependency rules?

[16:45 - 16:51] c05: Can you give me a guess as to what this task is about?

[16:52 - 16:53] r02: We can do that later. We'll probably say a lot more.

[16:57 - 17:00] c05: I think a visualization might help.

[17:02 - 18:38] c07: I think, especially when you get tasks like this, it's hard to get into the design. And that was maybe one thing I didn't get to earlier, but when you're usually at work, the projects we design could be better documented. I often have tasks that say 24 hours to do it, and I don't know what that means at all. It says that some emails should be sent automatically, based on some things that happen. It's very unspecific compared to what we just went through in your tasks. So the difference is maybe that when we actually have these very specific tasks, it's a little easier to follow and conform to the rules that are there. But at the same time, I have no idea what it's going to be used for. I don't understand the design choice at all to make this new logger instead of doing something else. So that might make it a little difficult, when you don't really know what the reason is that we're implementing these things. Because we haven't been involved in a design process, we haven't been the architects of it, we don't know what the architects have in mind. We just carry out the tasks like that. So it can be a bit difficult. Because you don't know if you're following the wish that was 100%, in addition to just following the requirements. But is following the requirements good enough, that can be hard to say.

[18:36 - 18:38] r01: So some kind of documentation that justifies?

[18:39 - 19:10] c07: Yes, maybe like that, why we're sitting here doing what we're doing, or what it's going to be used for. Because then you can keep the future in mind, and maybe it's me who's used to being in a place where it's not that big. There aren't that many employees, it's not big projects. You have insight and overview, and are allowed to take a lot of responsibility and decisions yourself.

Where here, I felt a lot that there are these rules, this has to be done, just do it. Hope that what I did followed the rules. And if I have followed the rules, then hopefully it is good enough. I am not used to working that way.

[19:10 - 19:14] r02: You are used to a slightly higher level, higher level of abstraction and all that.

[19:17 - 19:35] c08: I think the thing with visualizations, I agree, that is also what I started by saying. Even more strongly I would say that in the same way that we have unit tests, if you have architecture tests. Hard limits. If you could set it up, then it is very easy to follow, because then it is just like that...

[19:35 - 19:38] r01: So that restrains the workflow?

[19:38 - 20:08] c08: Yeah, like, you just get a fail. If you try to push something, you have this hard rule. The hardest thing you can do is set up a clean system, just like you can have linters or another system. That also falls into some documentation, they say, you don't document in the code, you document next to it. I think the best thing is as close to the code as you can have it. That would be cool. And then you can also visualize it. I think both things are good to have.

[20:10 - 20:16] r01: Now you both mention visualizations. There were pictures of the architecture in the documentation.

[20:17 - 21:01] c08: What I think of when I say visualization, and it's also good to have, is live documentation. So imagine what the changes in the system are like right now. Now I've added a new class, so I'd really like to see it pop in. Or okay, now I've made three import statements in this class. Boom, can I get a live update of that? Now I can see, now I just added some dependencies over here. Well, I know, in a larger project it's faster to grow this graph. And it's also difficult if we're now importing from, I don't know how many, 25 different NuGet packages. And then all of a sudden, and they're importing from something. What depth in that dependency graph? So that might also be something if you can scope it in and out.

[21:01 - 21:04] r01: So you're thinking of imports? Visualization of imports?

[21:06 - 22:47] c08: It's a place where you can see, you can clearly see that you depend on something. It's so static, you can also have some errors that happen, but it's not necessarily architectural errors, but you can have errors that happen in something or other, something you depend on, something that is dependency injected somewhere or other, then there can be some errors somewhere else in your chain, and now we're talking more about errors and not so much about architecture. But if I have to step back a little in relation to the work I do, it depends a lot on the work... which... project. I think for myself I like to keep a fair amount of structure, at least I tell myself that I like to actually refactor and keep things clean. But I would say my insight is that that's not what people pay for. People pay for software that works, and if you tell them that we can make it easier to maintain, you get less technical debt, you have to be a good sales person, and there are also some who accept that, but it becomes a trade-off. You have speed... Some of you said earlier that we had a deadline of an hour and a half. So it was like, what do you value more? Is it that you followed the architecture, or is it that you got the features done? So that's the trade-off that you're up against. So you could say that if you have some hard tests or linters, or whatever you're running on this, yeah, and then boom, it's not working.

[22:47 - 22:48] r01: So did you have a specific metric?

[22:48 - 22:54] c08: Yeah, exactly, but you could also say that then you can't ship your feature. And then it would be another place where you could...

[22:54 - 22:57] r01: Okay. Active part of the workflow.

[22:57 - 23:00] c08: Yes, if that was the case, you would... Could have put it in.

[23:01 - 23:02] r01: What were you thinking?

[23:05 - 24:44] c05: Personally, I like having a way to see... I had actually spent time on that at work and there is a script in Python for PL1 that can read your PL1 source code and then create, well, it's probably a call graph, but here it would be a dependency graph if you wanted to. You see, and so that I, as a newbie to the company, can figure out what the hell is going on in this code base. And it's pretty handy, especially if people have been a bit crazy about what they include, to be able to see, okay we're using this thing here, it's over here, and things like that. And then there's also the fact that it can suddenly become a very, very large graph. Because I noticed that there were some of these graphs that had 2,000 nodes, and there were others that had 10. And that's a bit, so you just have to figure out how to split this up? But something that gives a good overview of it is really worth a lot. You could say that a call graph usually has a bit more nodes than a dependency graph, but things like that are worth a lot, I think. And I would spend a lot of time making it work, if possible. But you're also right that people don't pay for cool code, they pay for it to work. And I know that from my work too. It's very difficult to convince a bank that they need to get off the mainframe if what's on the mainframe works. So, that...

[24:27 - 24:28] c06: So as soon as you say the prices, you scare them away.

[24:28 - 24:45] c05: Well, also just the uncertainty. So if their shit doesn't work one day, just one day, it costs a lot of money. It's not even so much the development work, it's more if it doesn't work.

[24:46 - 26:09] c07: I also just talked about how it's important to have good architecture. But in the end, I don't get paid to write comments or tests. And I haven't written any of that since I started as a consultant. It's just getting the code to work, and then we test it almost integration-wise. Now I'm developing a CRM, so it's a lot of sitting inside Microsoft, creating tables, giving them logical names, sitting and building context, making some plugins, developing on it, going in and checking when I create a new record that what's happening is supposed to happen. If it works, and the edge cases are fairly well controlled, then out to the customer, and then hopefully we've saved some time and not spent more, because there's often not a lot of time for those things. So in the end, when it comes to a specific day to day like this, we try to do it as quickly as possible. And then we don't focus so much on the quality of the code. But you want to. And it's a bit annoying sometimes. I've spent several hours on something that wasn't paid for. Where I just threw it on my own internal time, and then it

went a bit beyond my graphs and my stats. But then that's because I don't think the customer should pay for it, either that I've been messing around with something they didn't want. Although... it quickly becomes hours of interest in sitting around doing something nice, and you just have to get used to it ... doing it less and less.

[26:09 - 26:11] r01: Get used to what?

[26:11 - 26:18] c07: Sitting around doing it nice because I think it could be nice that it was nice when the customer doesn't intend to pay for it.

[26:22 - 27:21] c08: I do it for myself too. Now I'm almost straight out of college, where I think it's nice. I still think that with these work projects. I think it's cool. I like to do it cool, but then I just know that I'm taking it from another pool. That's the shift you have to have. Before, everything was... It was part of the university, no one pays me for X Y Z. So I do it because I feel like it. But now I have to get it mapped into a structure where it's me who does it because I think it's cool. No one wants to pay for the extra. That is myself... I want to have a better stamp of approval on this, and that's why I spend the extra hours on it. But the thing is, if you want to work 24 hours a day, you can do that too, but you only get paid for what the customer wants to pay for.

[27:21 - 27:30] r01: We don't have a lot of time now. I think we're already over time, but is there anything on the rope that you think is important to get on the record? No?

Listing 5: Treatment 2

[00:10 - 00:20] r01: What do you think about the tasks you just did? Just in general.

[00:24 - 00:44] t12: I think it was okay. I think they were okay well described, but I couldn't really see if I should think more than that. Because I could clearly see that there were some dependencies, but I ignored them.

[00:44 - 00:44] t11: What?

[00:45 - 00:52] t12: I ignored them. So ArchLens was like that. Don't do this.

[00:52 - 00:53] t11: Did they point in all directions or what?

[00:53 - 01:08] t12: Yes. Yeah, so there were dependencies in directions where there shouldn't be, but I don't think there was anything said about an interface in the bag, so I just kept going.

[01:09 - 01:37] t11: I did exactly the same thing and tried to implement it the way I wanted to implement it if I just had to finish. And then an arrow came in the opposite direction, and then I'm like, whoops, because I had a lecture with Mircea yesterday, who was like, that's not great, there has to be one direction on it, and otherwise you can use dependency injection or some other stuff like that. And then I'm like, oh, that was a lot of work, but I gave it a shot, and I think I spent an hour on it or something.

[01:39 - 01:42] r01: But you actually ended up...

[01:42 - 01:56] t11: Only having arrows in the directions that you had when you gave me the assignment. But I also corrected the tests and stuff to implement a logger or something there. So it was a lot of work.

[01:56 - 02:05] r01: Did you feel that you were influenced by what Mircea said in the lecture? Or would you have done it if it was at your workplace?

[02:06 - 02:38] t11: Well, I did. I think I was influenced by having the visualization that was given to me, so I think I was more influenced by not wanting an arrow pointing the wrong way. It almost became gamified. And of course, since it was yesterday, that lecture, there might be some attention bias. It was very new to me. So fresh in my memory. But it's hard for me to say how much it was.

[02:44 - 02:57] t13: Yes, I can feel that I've been working as a tester for the last few years. What else? If ArchLens could have screamed at me, I think it would have.

[02:58 - 03:02] r01: Were the tasks easy or hard or confusing?

[03:02 - 03:20] t13: It wasn't because they were super complicated, I was just a snot-nosed person at implementing them. Again, I've worked on things that rely heavily on deep frameworks in the C layer and the C++ layer.

[03:20 - 03:21] r01: So it was a bit far-fetched?

[03:21 - 04:08] t13: Yes. It was really nice with the visual feedback. Even if at some point it just changed the graph. I actually wanted to use the tool later. Because all I do is try to get things tested early. And this is a static test that doesn't require you to push things. There aren't very many of those. And there aren't very many that visually show it this way. That is, it's not a pain to sit and use it, so I would be able to use it in my daily life actually, to sit and push it to a shift left method.

[04:10 - 04:40] t14: I think I can nod in recognition of that. So, isn't that the thing about getting fast feedback loops what you crave most as a developer? Because there is so much build and there are a lot of expensive tests, so the thing about getting cheap tests, where it's faster to get some feedback, is always good. I struggled a bit too much with C#, I think, to get through, but I can see the idea of doing it.

[04:40 - 05:08] r01: Now I'm also thinking mostly about the bags, and not so much about ArchLens specifically, but yeah, try to see if I can put a few more words on it, also how it's like, well, let's take ArchLens, how was it part of your development, when and why did you open it, for the visualization?

[05:09 - 05:28] t11: I had it open over on the right side, and then when I thought I was done with factory-correcting all the files, I looked to the right and saw if I was done. So that's what I mean by Gamifying it. I wasn't finished until it pointed the right way.

[05:28 - 06:26] t12: I also had it open in the sidebar, so to start with all the time. And then, because I wanted more space on my screen, I closed it down, but then I opened it up every time I thought I had solved a sub-task. So when 1A was finished, I tried to look, and then when 1B was finished, I looked again. But I think I was a little unsure about the task description and the experimental part. how much

I had to go in and do more things than what was written in the task. Because in task 1, I did exactly what was written, but a lot of dependency issues came up. And it was very clear that yes, this is not good, and that was also written in the README. But I think I'm a little unsure about if that meant I had to refactor the whole thing so we could get some arrows pointing the right way through.

[06:26 - 06:27] r01: So what did you end up doing?

[06:27 - 06:33] t12: I just had a lot of dependency issues

[06:33 - 06:36] r01: So you followed the task slavishly?

[06:36 - 06:38] t12: Exactly, yes.

[06:38 - 06:40] r01: Nothing extra?

[06:43 - 06:52] t12: No. But I used it between each subtask and when the tasks were also finished to look again.

[06:52 - 06:54] r01: Would you normally do that in a workplace?

[06:56 - 07:09] t12: If I had been instructed in it, yes. If it was like a tool that was actively used, and we agreed that we use this tool because X, Y, Z, and if we used...

[07:09 - 07:20] r01: I'm thinking more about the tasks. So if there was, if it was described, such a task with sub-criteria, was that also roughly how you would add it, or would you perhaps have more freedom?

[07:20 - 07:49] t12: No, I think that if I could see that this didn't look good, I would do what was needed to make it good. So that was the thing about me being a little unsure about what the experimental part was, and how much freedom you had? How important was it that the dependencies went the right way? I think it was a little, not unclear, but maybe it wasn't completely clear to me.

[07:53 - 07:57] r01: How did ArchLens fit into your flow?

[07:59 - 08:10] t13: I just think I got to a point where I was like, now it's just there, don't look. It wasn't happy.

[08:10 - 08:13] r01: What do you mean it wasn't happy?

[08:13 - 08:53] t13: Well, it started rearranging on the box, so I had no doubt that it was pretty mad at me. I also just focused on getting the tasks done, but having visual feedback like that is really good. I just think it's a super good tool. I'm not used to working with development, so I just focus on actually getting something in.

[08:57 - 08:58] r01: Yeah, what about you?

[08:59 - 10:48] t14: The problem was that I didn't really have time to... I used it in the beginning in relation to this thing of getting an idea about, okay, there was this architectural ambition that the code should conform to certain things. And I could see, while I was sitting there and struggling with mine and trying to get my C# templates not to complain when I tried to test, that I would probably be installing something, or be in the process of implementing something that definitely didn't live up to what was the specification. So I slowly sat down and started thinking about this very thing, okay, how am I actually going to make it clean? What will that require in a C# context? So that was the thing about getting this, what can I say, assumption... It wasn't like you weren't made aware that right now you were actually on your way to implementing something that you would probably have to redo again at some point, or that would become a bit of technical debt at some point, if it was a bigger thing. So in that regard, the mission was accomplished in terms of the fact that it's not something you... Just missing... Often you have a lot of considerations when you're developing something. So it's not just about conforming to an architecture, it's really about: Does it work. Because you have a PO who's standing there tugging at you and saying: Does it work. Because I'm going to show it in five minutes. So it has to work and it has to be, you know, two seconds before we get that answer, or whatever. And then it's just really nice to have some tools that we agree on. This is not finished until these arrows turn this way, we agree. So, often, it is also to visualize these challenges to those who are sitting and are responsible for the business part.

[10:47 - 10:48] r01: Yes, which is non-technical.

[10:48 - 11:20] t14: Exactly, and we agree that not all code is equally good. That's also this whole AI discussion, that's also really down to this. Yes, it's great, we can make some code quickly. But are you willing to pay what it costs to have fast code? Because all the arrows don't turn right. So it's just a tradeoff. As long as we all agree that it's a tradeoff. And that thing with the visualization of: No, we have something where all the arrows are fucked up. But if we agree that this is what we sell, fine.

[11:20 - 11:32] r01: So what is your... Now it might be very subjective, but what is your position on exactly following architecture. Is it important to follow architecture? Yes?

[11:32 - 12:15] t13: Yes. Now I have sat as the one who receives all the things after the architect has been broken. And I am allowed to sit and play with it. And I would say, if you don't follow the architecture, then things just go wrong. There is no maybe, it will go wrong. Because there are a lot of things that behave funny as soon as you put together the whole system with all its integrations. And then there's something or other that you forget. Then there's some dependency that you haven't fixed. There's some shit that you haven't copy-pasted over, something is missing. And it happens all the time.

[12:17 - 12:19] r01: So yeah. What do you think?

[12:19 - 13:19] t12: I actually had a talk with some friends the other day about that as long as it's people who have to work with the code and understand it, it's important that there's an architecture that you can understand. But if it's only AI, where you have to prompt something to make a product that works, that sells, then it might not be necessary to have an architecture that's so understandable to it. I feel like we're learning a lot here, and I also agree that it has to be readable, you have to understand the architecture and the hierarchy that exists, but I also question a little bit that if it's not people who have to understand it, but it's just computers, and they work, then I can be a little, okay, but so what? I think it's quite interesting, that area or something.

[13:20 - 14:11] t13: You still have to remember that an AI is basically just a glorified statistician, so as soon as it doesn't follow an architecture, it can come back at you hard. And especially when we're talking about multi-agent workflow, they have to think as different instances of the same thing. So as

soon as someone starts to deviate from that, the others start to say, what's going on? And then they just take the most likely choice, and then they go off the rails. So it's actually quite important that it follows its own architecture, which is also why you have these, what should I say, rule files that you set up for them, where you tell them how to do it, and I don't think I've seen a single one of them yet where architecture isn't included in those files, because it's just really important that they follow them, because errors occur, and misunderstandings arise, especially between agents who have no human interaction, not to mention testing the work afterwards, which is hell.

[14:12 - 15:59] t14: But what's also the issue, as I experience it anyway, is that the AI-generated code that I've seen has been enormously convoluted. So it's simply so difficult for a human to keep in their memory something that has so many layers of abstraction applied, more or less just like that, I feel, standard. That we abstract just for the sake of abstracting and in reality maybe not so... That's how I feel AI works. And that makes the code incredibly difficult to show around on a human basis. And I would say, my feeling is, if you as a company are smart, you keep all your really business-critical things close to your body and let people, or at least be sure that it's human-understandable code. And then there can be lots of other, less relevant, derivative business things that you can maybe lay out and say fine, we'll just make that into some deep module, where we know that there are some APIs that are clear and easy to understand what's going on down there. But business-critical things... I'm just still not at the point where I think you can have some code that's as difficult to understand as what sometimes comes out of some AI. At least that's my experience. That I wouldn't want to be the one to sign off on, okay, that convoluted, very difficult to understand ping pong... I don't think so. But I noted that, what's the name of the Vibe-koder guy who made some blog post where he thought there should be a human in the loop who signed off on all AI generated code. And I just think that as a developer you have to have some clear requirements to say that I can't be responsible for something that is so little humanly suitable for understanding.

[16:01 - 16:16] r01: Let me just try to time-box it a little bit, because we don't have that much time, so I'll quickly jump on to the next one. And were there situations where you deliberately chose not to use ArchLens, or where it didn't help, and what situations were they?

[16:19 - 16:34] t12: I used it all the time, but as I said, I didn't act on the dependency issues that were there. But I used it all the time. There weren't any situations where I didn't...

[16:34 - 16:39] r01: So it was more that you ignored it if it was. And then also to get some extra screen space.

[16:39 - 16:46] t12: Yes, and then open it up again.

[16:47 - 17:06] t11: I suspected that it looked at imports and usings and those keywords. So I tried writing the entire path to the class that I want to access. Directly, inline. And when I did that, the arrow was gone, the bad arrow. That's also a solution if we only look at these graphs here.

[17:07 - 17:08] r01: Okay, yes.

[17:09 - 17:20] t11: Because I didn't import, I just called directly. Okay, now I understand how it works. This is clearly not the intention.

[17:20 - 17:30] r01: But it's very funny, because I think I actually said it as a flag that you have to have the right one... You can't have unused imports when you build.

[17:32 - 17:36] r02: This is a bit of the opposite situation. Instead of using an import, you say that I would like to use Infra dot something or other.

[17:36 - 17:42] t11: Yes, from this folder, or from this folder. So I use the namespace directly to access ...

[17:44 - 17:58] r01: Allright, but this thing about having this visualization that ArchLens gave, did it change your way of thinking about the dependency policy? Or did it mainly just confirm something you already knew?

[17:58 - 18:22] t12: Well, it changed 100% for me. So in the first assignment, where I kind of solved it, and then I thought, now it's done. And then I look at ArchLens and say, well, there's something wrong with the architecture. I wouldn't have spotted that if I didn't have the tool. So I just thought, now everything is cool. Yeah, so it definitely did that for me.

[18:27 - 18:39] r01: It was more or less the same, right? Consensus? Yes. Okay. Cool, so we'll just iron it out.

[18:39 - 18:40] r02: We have the last 5 minutes.

[18:40 - 18:59] r01: I think it's fine. Would you use this in a real project? I hear a bit of yes. Yes, but primarily to be able to, for some communication maybe, and understanding, or what, am I wrong there?

[19:02 - 19:06] r02: At least there's talk about using it as a team tool, right?

[19:06 - 20:06] t14: Well, that's definitely a context of use that I could see it in, again because this thing about it being extremely difficult to communicate is my experience, what can you say, functionality isn't the only parameter. And that's just... I just think that's how the world is. Somewhere, and you can quickly become a bit of that developer who sits in the corner and snipes at something. Where in reality it's just... These are pretty important things to get right. It's a bit like coming out and saying: I would really like us to fix our legacy code. That's not something that gives promotions, right? It's just when there are layoffs and things like that. Then it's not the people who sit and make the newest product who are smoking. It's the ones who sit and make legacy. You know that sometimes you can be a bit of a minority if you sit and want to make proper architecture. Yes.

[20:08 - 20:53] t12: Agreed. And again this thing about it giving feedback quite quickly, so it's very easy to use, and it updates in a few seconds, and every time you type in the code, I think that's nice. I definitely think there's more it could do. For example, I missed it a bit that it came with some recommendations or something like, "Hey, you have this arrow going the wrong way." But that's probably because X y z. But of course, that's because it's not a fully mature tool, I assume. But it would be nice if it were also smart enough to make suggestions on what to do to fix the arrow. Then I think it would be even more useful for me.

[20:54 – 21:21] t11: I think it didn't highlight if you had an arrow going the wrong way. They just looked like the other arrows. But if we have a set of rules... I mean, the company had sat down and made this set of rules, then it might as well be red, that arrow. There are good arrows that are green, and there are bad arrows that are red, or something where you can see what kind of work needs to be done? Where do I need to work on? Because then I can click on those arrows and see where the code is that points in the wrong direction.

[21:23 – 21:43] t13: For me, it's missing a bit, as you also say, the analysis part. Is it okay, or is it not okay, that the arrows turn the way they do? Because it will be dynamic from project to project, that you jump around in. So it's missing something like that, well, it deviates from the normal pattern. Because if it does, then it shouldn't.

[21:43 – 21:46] r01: Okay, so you think more about forcing it, if it is?

[21:46 – 23:03] t13: No, it's just supposed to visually provide feedback that this isn't how it usually looks. That's not how the rest is architecturally designed. Be aware that it can cause problems. That's just how it's flagged, because it's not supposed to be a single source of truth. It's just supposed to provide quick information to the hoof, this doesn't follow the thinking that has been in the rest of the program. And where I really see that it can provide something is when you start making major new features, or you have to onboard a new employee into it, who is going to try to make their first addition to this. Because when you have a new employee, they can actually sit and look at a graph, okay, the arrows point this way. That's how it's designed. Well, then try to follow it down and see how things are built up from there. I mean, especially event-intensive WPF applications, this will just be a pyramid. And there you will actually be able to follow it down through, because it's pure hell to go down through such an event-intensive WPF application. So there it could really be good. But also for when it's big features, that you don't tie yourself up in knots. especially with AI development, if you just use some kind of auxiliary tool to just generate something quickly, then poof.

[23:03 – 24:03] t14: It's also that observability that's kind of missing in relation to the whole AI dimension, where it just explodes just in advance to what it is that we get out. And I just think that you can get that feedback pretty quickly on, okay now in this sprint, how much is our technical depth in reality on this, how has it moved? And you get feedback on that pretty quickly. And it's not something where you have to ask for extra resources. Now we just have to figure out how this has affected our entire infrastructure. It's super hard to bother with, I think, and do, but maybe with a few indicators you could just get, you know, rounded troops up. So continuously. But there's still something missing... Many of the metrics we work from are lines of code. These are metrics that are really very bland in some places.

[24:04 – 24:05] r01: I just want to give you the opportunity to say...

[24:05 – 24:18] t12: It was just because I'm going to leave in a minute. It was just that thing with color-coded errors. The thing about it being black arrows right now. It also meant that you just had to remember what it looked like at the beginning.

[24:19 – 24:21] r01: So you didn't use that Compare to Remote?

[24:21 – 24:29] t12: Yes, I did, but it kept having the wrong ones... Nah... Nothing.

[24:29 – 24:35] t11: I didn't use that at all. I just used that very simple view.

[24:37 – 24:37] t12: Me too

[24:37 – 24:45] t11: And that compare with remote, it looks really good. Too bad I didn't use it. I haven't seen it before.

[24:45 – 24:46] t14: There's UX feedback.

[24:48 – 24:52] t11: And now the arrow turned green, because things have been added, I assume

[24:52 – 25:06] t12: But it's just as much about adding things, but that it's going the wrong way. Yes, that's illegal. Yes, the rules you had written in that README, that it also showed that.

[25:06 – 25:16] t11: But now I can see that I've added some dependencies, and then it turns green. It's perhaps a bit misleading that green means added more, i.e. more dependencies in that direction.

[25:16 – 25:19] t12: Yes, but that's fine.

[25:19 – 25:35] t11: I don't associate adding a lot more with green. If I remove a dependency to clean up, it will turn red. But why red, it can still be nice

[25:35 – 25:45] t13: You have to measure it against the big picture. How do the dependencies work? Overall and then measure against... green, yellow, red

[25:45 – 25:52] SPEAKERS: (Participant leaves, we say goodbye)

[25:52 – 26:02] r02: We would like to hear more. But that's what we had.

[25:58 – 26:00] t12: Hope you can use it for something.

[26:00 – 26:02] r02: We definitely can. Thank you so much for your help.

[26:02 – 26:12] r01: Are there any closing comments? I think this is actually a really interesting discussion.

[26:12 – 26:37] t13: Yes, in relation to the sprint thing, I actually think that this should be used much earlier. So, it should be an everyday basis that it gives feedback on this. Because you want to sit around all the time and avoid doing these annoying fuck-ups. And when you think sprint level, I start thinking about other static analysis factors that take the whole thing and run it through.

[26:37 – 27:10] t14: It's like to get that in as a team and kind of look at it again, just to get that feedback on, okay are we moving in the right direction, because you often get that person telling you, okay when you go in and work on some code, leave it nicer than you found it. That's the classic assignment you get, when you're sitting outside and you ask, can I refactor it? No, you can leave it nicer than you found it. Yeah, okay, fine. But then again, it would just be nice to get some feedback on, okay, have we as a team actually moved the needle one way or the other in relation to the debt we now have? Because that's what you do everywhere.

[27:12 – 28:41] t13: Here I see it more as, this tool is super good for catching things early, and then you can apply other static analysis tools to when you just start putting the code together from different sides, to actually analyzing the entire repository at once, giving you reports with a greater level of

detail. This is great for constantly being informed about what you're doing now, but it will most likely get cluttered if you start asking it to do both, because you get a big report out of many of these static analysis factors, so I really feel the value is good in keeping it at this point, with it really just being, what should I say, a fly on the wall, keeping an eye on it for you and tapping you on the shoulder if you start doing something that you really shouldn't be doing, or just remember to double-check this one because you're starting to move into a bit of wobbly terrain. But there are a lot of static analysis tools that actually give you exactly the report you started talking about. That's what most of them do today. And the problem there is that they give it to them too late. Because by that time it's finished. The feature is finished, the bug fix is finished. It's done, no one is going back to it. And that's the problem. It has to be used in conjunction with the tools, so you avoid it before that. And then you can use the report from, what should I say, the common industry standard static tools. If you're going to break with something, there's no point in breaking with the ones that are already big on the market, that the bigger companies use, because they don't switch away from them, I'd just like to point out.

- [28:41 - 29:08] t14: I've rarely experienced, I've actually never experienced that where you, again, you have lots of reviews, you have lots of functional reviews, but it's really rare that you delve into, how has the architectural quality of what we've done moved things? Have we actually delivered anything where we haven't delivered more hours of work to ourselves down the road?
- [29:08 - 29:11] t13: And that's exactly why you need to use them in conjunction with each other.
- [29:11 - 29:18] t11: But I actually think you're right that for stakeholders it would be really good if you had to communicate to someone who isn't technical.
- [29:18 - 29:20] t14: We've got all this functionality, but we've got all these arrows pointing the wrong way.
- [29:20 - 29:28] t11: Exactly. If the arrows are red and green, and there have been ten red arrows since the last time, but you've got your features.
- [29:28 - 29:32] t14: Yes, exactly, but you've gone into debt. So you just need to know that.
- [29:33 - 29:39] t11: Exactly, so what do you want to do now? Should we make more features, remove three bad arrows or...

Listing 6: *Control 3*

- [00:03 - 00:15] r01: That's it. Alright. Well, would you like to put some thoughts on those, or some words on the tasks you just did? What do you think about them?
- [00:16 - 00:26] c09: There are some that are significantly easier than others. But I also think there was a little cheating in terms of how we best maintain... Do you want it in Danish or English?
- [00:27 - 00:28] r01: Danish is fine.
- [00:29 - 00:44] c09: How do you best maintain the current architecture, because I was... at a point when I started having references from application to infrastructure, where I was like, mmm, I'm cooked. That's probably fine, bang! And then I went back at some point and saw that it probably wasn't fine.
- [00:46 - 00:51] r01: Why do you think there were... What did you call it? Traps?
- [00:51 - 00:57] c09: I think they were traps because some of them are unassuming changes that suddenly break with the architecture.
- [00:58 - 01:01] r01: Okay, and what makes you say that it broke with the architecture?
- [01:03 - 01:13] c09: In relation to the fact that we have a clear separation, of among others application and infrastructure, it's this thing with reference directly between them, rather than through this, i.e. the semi-layer, domain, where we kind of use the two things.
- [01:13 - 01:20] r01: So it's like, so because you read it in the README or something? What about the rest of you?
- [01:22 - 02:36] c13: I had a bit of the same feeling. So now I can see that you have tried to follow an Onion architecture. So with some of the things... I also think in a business setting, if I had done this in the company, I would probably have approached it a bit differently. Also the thing about having to discuss with your teammates. Because there were some things where I didn't think it should have been in the layers they were. So I would have probably done a refactoring first. Or asked how we could solve this best. And then there was, for example, also this, if you had set up some Dependency Injection. So I could abstract away some details. For example, something in the application. I think that was the last task, I would have really liked to have put an interface in the application, and then implemented it in the infrastructure, for example. And I felt a bit like, okay, I have to do something now, and it's not that big of a system, etc., so I have to compromise a bit. So that's also a bit like, I'm making invisible introductions of technical debt, I would say. Because, obviously, I'm not going to break those boundaries, but I'm putting it in the wrong layer a bit.
- [02:37 - 02:48] r01: Yeah, okay. But how did you keep track, because you mentioned that you started to like... Well, you could see that you were breaking them. How did you see that?
- [02:48 - 02:52] c09: Uh, I sat down and started checking references.
- [02:52 - 02:53] r01: References? So imports?
- [02:53 - 03:06] c09: Yes, so I've used AI for that, where I've kind of fed it the context and the README. I've made a mandate for what architecture it should follow, unfortunately AI isn't that good at following architecture, but it was kind of made aware of, uh, that something is starting to happen with some architecture, so if that's the case, then I have to sit and read references.
- [03:07 - 03:11] r01: Okay, so when you say references, do you mean imports?
- [03:11 - 03:12] c09: Imports, yes.
- [03:12 - 03:35] c12: I think I started by doing things quite manually, also in that logging, that was definitely the one I spent the most time on. Where I could see without using any tools or anything. I can't just call in the wrong direction in relation to this onion structure. But I can't remember how to do it if I have to get around it.

[03:36 - 03:37] r01: How could you possibly see that?

[03:38 - 04:17] c12: I'm sitting with the README on one side of my screen and I can see, well, but I'm not allowed to have an import that way. Then I can see that I need some functionality, like a logger class I just made. Which I'm not allowed to call. And then I talked to an AI about how to get around it. And I ended up having a lot of doubts about how in-depth you should be, how clean it should be. So I ended up both changing some tests and making a fake logger thing to run the tests. And I couldn't even judge if it was too much. But.

[04:17 - 04:25] c11: I thought those architectures were examples of how you could use your tool. So I was like, that's cool, and I didn't even think that we were supposed to keep it.

[04:30 - 05:11] c10: I kind of have that approach where I'm like, even though it said you had to follow it, sometimes there were tasks that made it impossible. Like that logging thing, because you have to have it in infra. And then you have this class in domain that has to call it. And so my approach is... Damn! Alright, you're the ones paying me, fuck it. And then just go anyway. Because that... Is it stupid? Yes. Is this system... I don't know this system. I don't know, it might be possible. So if this is the task I'm given, then I also assume that it's already messy enough. If these are the tasks that belong to the system, then the system must have a corresponding level of messiness. So that's why I'm...

[05:11 - 05:18] r01: So the description of the tasks weighs more than the description in the documentation?

[05:19 - 05:21] c10: Yes, I would say that.

[05:21 - 05:46] c13: I also think that was the thing with the fact that now there were different folders called domain, application and so on. What if they had been different projects, so that the compiler says that you don't need that. Or if you want to create a reference, then you at least actively think about it as a developer. Where the folder thing, especially with AI, it can just go fast when you go the wrong way, right?

[05:49 - 06:36] c10: I also think that I don't know. It's also that if you get a task from someone around, that you have to do something. I don't know. It's a bit difficult, because in these tasks sometimes it's impossible. Then it's because I agree that I would talk about finding a way to restructure it, but when you have an hour and a half. It has to be done. Then there's a lot of context within it. Okay, I have time pressure, so it's probably some kind of deadline or busyness. There are times when it's such small tasks that have to be done. It's probably some consultant work. And then it becomes a lot just like that, it doesn't matter. It also comes from my own experience, where it's just fucking architecture at my old job went out the window, who gave a shit.

[06:36 - 06:58] c12: It's funny because I've had exactly the opposite attitude, and it might also come from the fact that at my job, there are those who are very detail-oriented, because it's something that's very, very important to a lot of people. So the thing about, okay, if it says in the documents, I looked at it many times, like okay, what are the rules? What can I call what? Okay, like that. That was the first priority, so it didn't matter how far I got.

[06:58 - 07:02] r01: So you see the README and documentation as rules?

[07:02 - 07:07] c12: Yes, so it's non-negotiable. I could never think of not trying to follow that.

[07:07 - 07:12] c10: It's funny, I'm doing something a little different. There are people who have tried to set up some rules at some point, and I can see that they've been thrown out the window.

[07:14 - 07:18] c12: But I don't know if I did it right either, but it was more like it was important for me to try.

[07:19 - 07:53] r01: All right, we're already talking a little bit about that, but let's try to see if we can stick to how you approach this dependency policy. Well, there was something about looking at some imports. How did you have it... did you have it in mind while you were developing?

[07:41 - 07:50] c11: I thought that document with what you did was if you actually wanted to make real changes to your code. And there I was like, I'm not supposed to, I just have to do some tasks. And so I didn't take it that seriously.

[07:54 - 07:55] r01: No, yeah.

[07:56 - 08:03] c09: Yeah, I just fed it to a machine, then I was like, go nuts, and then at some point I was just like, I hate you, you've gone nuts.

[08:07 - 08:35] c13: I actively think about it, so now I'm kind of familiar with onion and clean architecture, etc. And it's really something I think about in my work too. So it's a bit like, I want to say an occupational injury, I can't really help it. So I think it's so active that I thought about it, but I also feel like I actively made a choice to compromise with it sometimes.

[08:35 - 08:36] r01: To follow the tasks or what?

[08:36 - 08:55] c13: Yes, it's probably a bit like c10 said, I actually really would have done it like c12. So the thing about, you know, doing it properly. And I also caught myself thinking, this, if I should have done it properly. So I would have spent an hour and a half just on this. Well, but I knew, I'm going to have to, so you just have to... Compromise on some things.

[08:56 - 09:00] r01: Would you rather it had been forced?

[09:00 - 09:01] c10: What do you mean?

[09:01 - 09:21] r01: So that, what is it called, that thing that you mentioned there, that project structure means that you can't have circular dependencies and stuff like that. Would you rather it was set up with project structure, and it was just hammered into your head that you can't do that?

[09:21 - 09:50] c13: I think so, because now I work in a team. There are a lot of junior developers, and there are also a lot of seniors. Today there are almost 20 of us in that team. If one can abuse it, then you abuse it. So that's just the rule of thumb, so you should almost have an architecture that supports the changes you're working on, because it's going to come through, I feel like saying no, if you've made a bad decision. So I would prefer that.

[09:51 - 10:23] c09: I would really like it to be like a compiler warnings thing. We started out with 4 compiler warnings, and what if there were 4 compiler warnings that said, you're breaking the architecture here, do you want to continue doing that, because sometimes it might be necessary, because architecture is not a one-to-one absolute truth either. Most of the time it is, but there are some

examples where you have to break it a little bit, to support some other good ideas. So it would be really cool to have a build-in architecture analyzer.

[10:24 - 10:53] c12: I don't know how relevant this is, but at my workplace, we use a very, very rigid, like, I don't know if it's a Solution error, or where these rules are, but it tells me that you shouldn't use this type of variable. Like this, I get a lot of warnings if I don't do it exactly as they want it. I think that works pretty well too. I don't know how at what level it works, but it's built into the C-Sharp program project.

[10:57 - 11:29] c10: Something that fucked me up a bit was that infrastructure was deeper in... Sorry, application is deeper in than infrastructure was. I wouldn't normally do that. Because I'm like that... So in an architecture, because I'm fucking... For example, we were going to make a repository, where do you put that repository? You don't put it in domain, you put it in infrastructure. But application is deeper in the onion layer. Which is... I just tried to bring up some other projects, in BDSA, for example. That's not normal. That's fucking strange. At least that's not what I usually see.

[11:31 - 11:55] c13: Yes, but that's just... Well, it's a funny architecture. You also have to catch it. And that's exactly the thing about it being clean. The thing about application being a use case for the business layer. And that thing about infrastructure kind of couples the implementation with this abstraction. So, that's what you can replace as you want. So it's that you have to use... You have to catch it.

[11:56 - 12:21] c10: But the funny thing is, even though there's this infrastructure-application-domain. So in the other dependency files, infrastructure and application are on the same layer. They just don't point to each other. They just both point to domain, so there is double confusing or incorrect documentation. And that was also something that confirmed for me that this is a messy architecture system. It just doesn't fit. It's some legacy documentation that doesn't work.

[12:21 - 12:22] c13: That's also true, you could do that too...

[12:22 - 12:45] r01: What's it called, how did you check? I hear something about import statements. If you even adhered to the architecture. You've also mentioned it a little bit, but... Did you refer to the README a lot, so did you look back at it a few times?

[12:46 - 12:51] c09: Yes, a few times, but no more than that. 1-2 times.

[12:51 - 12:52] r01: 1-2 times.

[12:52 - 12:55] c10: I mostly looked at that docs folder. Where there were 3 pictures in it.

[12:55 - 12:58] c09: I looked at that one too, it was good.

[12:58 - 13:03] r01: Will you do it normally? I mean at work? If it wasn't such a... newer project?

[13:03 - 13:08] c09: We don't have that. No README, no architecture. Only vibes.

[13:11 - 13:22] c10: At my old job we were fucked. There was nothing. So you know, whatever structure you made was more the structure that was there when you started. So I don't know.

[13:23 - 13:35] c12: At my workplace there are a lot of protocols for everything, so I always have open on another screen, okay, next step in my to-do, of getting things into production. So I sit there all the time and look at it.

[13:37 - 14:10] c10: It depends on the culture you're in. If I was at a workplace where I was told that remember the most important thing is that you keep these architectural things that are in the code in advance. Rather than the tasks you get from, you know, Karsten from the fourth floor, who has never opened a computer himself. Okay, fine. But if it's something like that, where these tasks are very strictly defined by the customer's IT consultants, they just have to be adhered to. And then, whatever documentation, it's just whatever. So that's how it is, so you do it.

[14:11 - 14:16] r01: What would have made it easier, do you think, to adhere to the architecture? You're saying something about it being forced?

[14:16 - 14:44] c13: Yes, exactly that thing about if you had it out in different projects. And then I also thought that you had introduced some Dependency Injection. So you saw... that, that, being able to say that we have the interface here in the application, then I can implement it in the infrastructure. Then it's certain that you...

[14:44 - 14:47] r01: So you think that should be in the tasks, it's very task-specific.

[14:47 - 15:07] c13: It was very task-specific, but still that about laying the foundation for it, so I just had to say, as scoped, boom boom, something or other. That's when I thought, it was one of those things where I was like, if I were going to sit down and do this for a company, I would say: Shouldn't we just start with this before I do this?

[15:08 - 15:08] r01: Yes.

[15:09 - 15:22] c12: Very low-practical in my experience, it is also easier to comply with rules if there are existing examples in the code. So the more established and streamlined your code base is, the more automatically you just copy that structure.

[15:23 - 15:54] c10: I also think that for me, the thing about tasks being so tightly defined is that there is not much room for maneuver. This one, which should be a logger class in this folder, this file, which is in another folder. But it doesn't say that. Should call it here. So it's a lot of, well, okay, it doesn't say that... It's so tightly defined that the assumption of, okay, but then I have to move this around, and things like that disappears a bit. Because then it would probably be there. So it suddenly becomes obvious that it's just breaking the architecture.

[15:55 - 16:08] c13: Yes, I've had that feeling too. And then I also wanted to move the small modules around, because then I'm like, no, I can't do that, I don't have time for that.

[16:08 - 16:10] c10: But it's also that then it just becomes a bigger refactoring task.

[16:10 - 16:11] c13: Yes, it does.

[16:11 - 16:32] c10: Which I agree with, it should just be a, okay, the way you've set this up, these five tasks I've been given are impossible. With this, where there's such a thing. So either way I just wanted to ask, okay. Dear project manager, should we do this or this? Either do a refactoring first, or I should just flush everything down the toilet.

[16:32 - 16:52] c09: I think it would have been easier for me if I had an overview of what the architecture

is like right now between each commit. For example, this code I'm about to push now. What new dependencies am I introducing? Okay, now I can see that there's an arrow between application and infrastructure. Maybe we should look into that more closely. So, something like that, these tell-tale signs of, okay, something's broken here.

[16:53 - 16:58] r01: So every time you commit, there's some kind of overview?

[16:58 - 17:10] c09: Potentially. So where is it, this, this is your commit from today. You can see here, you've introduced three new dependencies between application and infrastructure. Fix them.

[17:09 - 17:11] r01: So something like a warning again.

[17:11 - 17:20] c12: Do we agree in C-sharp that if we were to create a dependency, an illegal dependency, it will always be an import line. You will always be able to see it that way, or are there other ways?

[17:20 - 17:32] c10: Not necessarily. For example, what c13 was talking about was that instead of having everything in a project just like that, divide it into projects so you can't just say import this into the file.

[17:32 - 17:37] c12: Well, yes, but if you have such an illegal dependency, you will be able to see it in your import.

[17:37 - 17:38] c10: You won't be able to import.

[17:38 - 17:45] c09: I actually think that you can refer to them specifically through namespaces, so that you don't import it, then you make a single reference to it once.

[17:45 - 17:46] c12: Right, right, right.

[17:46 - 17:55] r02: Technically, instead of saying using CoDepend.Application, you can just say where you need to use the method CoDepend.Application.UseCase.Run.

[18:04 - 18:21] c13: I'm also thinking about that if you could get such a visual overview when you make a pull request, for example. Then you're like, ugh, okay. Because it almost has to be visualized for you, or so that you can become aware of it.

[18:22 - 18:23] r01: Should there be some kind of alert?

[18:23 - 18:34] c13: Yes, and I feel that when I've made a pull request, I'm a little more receptive to taking it in. Whereas I feel that when I commit, I'm sometimes a little in the flow. Yes.

[18:34 - 18:40] c09: Yes, that would actually be the coolest place to be.

[18:40 - 18:51] c10: I would also say that it's like this... Just in the PR that an automatic message comes up that says... Well, just a, I don't know if you can call it that, or just like checking, and just saying hey, you're no longer following the architecture. It's just a test that fails.

[18:52 - 18:55] r01: Yeah, okay, so just a test like this...

[18:55 - 19:00] c10: A test that just fails to say, you have introduced a dependency between these two things that does not follow the architecture.

[19:00 - 19:11] c13: Yes, and I also think it makes sense in terms of having teammates who are going to review. They might also have a better insight into, okay, how can we do it in an even better way, than sometimes you yourself, right?

[19:12 - 19:20] c11: I think most people don't want to sit and code review imports. Unless you see that changes have been made up there. Then you just go to the code a little faster.

[19:20 - 19:43] c10: You could also show that, at least I feel that way, to a project manager. Try looking here. If I show you that it says using this, it doesn't mean anything to you. But if I show you in this PR, which you might have a little more understanding of, that there's a big red cross that says NO. Then you might say, okay let's rephrase it. Because otherwise I feel that if there aren't any checks like that, and it just says green all the way, then that's it, what's the problem?

[19:46 - 19:52] c12: I totally agree that on a PR it's a really nice and convenient place to have that type of information.

[19:54 - 19:59] c09: That's also where you can catch that this is a problem, because as soon as you get your PR in, you're cooked.

[20:01 - 20:02] c13: Yeah, that doesn't happen.

[20:02 - 20:15] r01: Don't you think it would be annoying that you've spent three hours on a task, and then you make a PR, and then you're told that you've now put logs in infrastructure?

[20:15 - 20:19] c09: It could be PR with the possibility of a commit basis.

[20:19 - 20:37] c11: So if you can just call something, if a PR probably just calls some kind of linter, or something like that, and you can just call it yourself. Before your code, it's just like, I know this linter is coming, let me run it myself first.

[20:37 - 20:42] c12: Of course, it can also be before, it's more a question of something like that not being allowed to come in.

[20:42 - 20:47] c11: This is the last time, and you are allowed to have the tool to do it before if you want.

[20:47 - 20:58] c10: Also because, yes, it's annoying, but it should only happen once. You do it once, you're told you're not allowed to, and then you've learned. But all of this, it also depends on if you have a job that uses PRs.

[20:59 - 21:05] c09: What if we just commit directly to main?

[21:05 - 21:06] c11: What if you don't have main?

[21:06 - 21:07] c09: Master?

[21:08 - 21:12] c11: What if the branching strategy is that branches should live longer than two years?

[21:13 - 21:15] c10: What if there is no branching strategy?

[21:16 - 21:16] c11: Yes.

[21:17 - 21:18] c13: I can't even think that far ahead.

[21:19 - 21:20] c09: Shoot to prod!

[21:20 - 21:24] c12: I'm getting real chills. I'm having nightmares tonight.

[21:25 - 21:34] c10: I'd like to point out that I was the first person at my job to do a PR. In its entire history, and this job dates back to 2006.

[21:36 - 21:38] r01: Well. Do you have any other comments here on the fall rope?

[21:38 – 21:48] c12: Well, in my experience, it's really nice with visual things like that that are generated. Well, compared to just a lot of text, but maybe just shimmer.

[21:48 – 21:56] c11: And if it's text, then emojis in the text. Good big, yay, green. Or red, bup.

[21:56 – 21:57] c09: Some visual cues.

[21:57 – 22:00] r01: So not just warnings. And if warnings, then with emojis.

[22:00 – 22:06] c11: Yes, then something or other, or you can color a warning like that red, like that.

[22:06 – 22:07] c10: Did you use dotnet test or dotnet run?

[22:07 – 22:08] c12: Yes, you should.

[22:08 – 22:18] c09: Quite often. Just sitting there fucking firing them off. At one point it started throwing a lot of exceptions, but all the tests still passed, and it could still run and run and compile, so it's probably fine.

[22:20 – 22:26] c12: Every single task, before I created it, and after I created it and ran it, I ran all three commands we had been given.

[22:27 – 22:58] c10: I honestly think some of the tasks were so simple that I forgot to do it. I'm going to do this one, it returns null. If it's null, which it is, then it does what it used to do. Maybe that's just something I assumed it compiled. But I think it was the four with dotnet test... I was just about to make a pull request, wait a minute, go back, dotnet test, everything fails. All right. Okay. I made two periods in a row.

[22:58 – 23:10] r01: All right, well then we'll stop... Is it something else? Then we'll stop recording.

Listing 7: Treatment 3

[00:05 – 00:26] r01: So what's going to happen now is that we're just going to do a debriefing. What just happened? Well, first I just wanted to hear if you could put some words to the tasks you did. How was it? What do you think?

[00:29 – 00:29] t17: Uncomfortable

[00:31 – 00:32] r01: Why uncomfortable?

[00:32 – 01:02] t17: I don't know... It's not something I usually focus on with work. Also because I work with Salesforce and it's a lot of drag and drop in a lot of cases. And a structure that I'm also used to now in terms of code. And then getting into this, you should first familiarize yourself with what exactly it is, how the structure works. And so I knew that I came in with a mindset that was like, okay I can just add this. And I just couldn't do that at all. And so I was like okay...

[01:02 – 01:04] r01: What do you mean specifically by that?

[01:05 – 01:16] t17: In terms of me thinking: So I can just add the logger here, which is requested in infrastructure, okay I can just refer to it here. And then it's like no you can't.

[01:16 – 01:17] r01: What gave you the impression that you couldn't?

[01:18 – 01:40] t17: Because it said very specifically in the readme that let's try to keep the architecture that is there, so that domain and application can't refer to the infrastructure folder. And it did that for me, so I got the idea that I can't. Or at least, I'll try to avoid that.

[01:40 – 01:41] r01: How did you notice that?

[01:43 – 02:21] t17: I ran ArchLens and I could just see that there were some arrows that weren't there before, that I didn't like were there, just to run an overview and see what exactly I've changed in what I've added. Instead of just adding stuff and then building it and being like, yeah, it looks nice. So just making sure that I hadn't changed anything. I had. And then I also had to find a way to work around it. And that's a mindset I haven't had in a long time. So it was just something you had to get used to.

[02:21 – 02:45] t15: But it was pretty cool to have that overview of the graph, because I haven't worked with it myself for a while. So it was like, how the hell was that right now. So I had also just implemented the changes, looked at the graph overview afterwards, and then I could see that there was clearly a dependency violation. So that was nice.

[02:45 – 03:03] t16: It's also the same thing, I'm used to a very messy, but at the same time there's a lot of structure in what we have at work. So it's very easy to say that I need to expand this, I need to add this, this belongs here. It's very easy to know where it belongs, and not so that it goes across everything possible.

[03:06 – 03:07] r01: In what way? How is it easy to know?

[03:08 – 03:47] t16: So we have a download handler, for example, and there we have, if it's an update, a new file, if it's a rename, if it's a file that's been moved, then it's spread out into different parts, structured. There's a lot of it that's already been put in that form, at least when it comes to that type of structure. We also have some things where there's no structure in it yet, because it's more in a prototype environment. So it was just one of those things, when you looked at the graph, that I broke this. Yes, figure it out again.

[03:48 – 04:03] r01: But you also had the README, which had some pictures of what the folder structure looked like. Why do you think that wasn't enough? If you think that wasn't enough?

[04:03 – 04:25] t16: I quickly forgot about it. I just created it and out of the corner of my eye you could see that something is wrong over here. I can see I have someone pointing to it there. I know that's not wrong when I look at it. You were reminded of it. I think if you had it closed and didn't see it, I don't think I would have noticed it.

[04:26 – 04:42] t15: It can also be difficult to have the mental mapping of dependencies when you suddenly have several classes and files talking to each other. It's super nice, actually being able to see it.

[04:43 – 05:30] t17: I also think it has something to do with the fact that you know, of course, even when you talk about things, it's much more at work. The longer you've been on something, the easier it is to have that overview, without having to use any visualization tools. And I just believe in my experience with the documentation that is sometimes created, where it is the case that the documentation is fine. And it is very fine that there is a lot of text. And for example in the README and things like that.

But it also just makes it so that when you need some information quickly, it quickly becomes very difficult to find, which I think is why I resort more to ArchLens rather than looking in the README. Because in the README there is some text that takes up space, there is no bloat as such, because it should be there. But when I just need to know if I've violated a violation, I don't want to go through the README with any text. Then I'd rather just pull it up and be like, here's what you've violated, thanks. Back to looking again.

- [05:31 - 05:45] t15: Especially because it breaks your flow of what you're doing right now. If you're suddenly in a good coding rhythm, you don't want to go into some long documentation and read it all through, it's better to just have a quick overview.
- [05:45 - 06:10] t17: Yes, I think it might help if you have multiple screens. If you're used to coding on multiple screens, because if you could just have the README open next to you, in a location where you know this is where I'm going to look, it goes faster. But I put, for example, notice that when I had ArchLens open, and I had the code open. So on my small screen I could hardly see a damn thing. Well, I could probably resize some of the things, but I didn't have the strength to do that at the time, so it's just a little bit, okay now I have half a screen to work with. So I don't want to have the README open next to me in some of the other 17 tabs I have open.
- [06:10 - 06:21] t15: I was also thinking about having ArchLens open just on a screen next to me, so you can always see, okay suddenly there's a violation.
- [06:21 - 06:31] r01: Okay, so that actually moves us a little bit into the second question. How was it part of your development? When did you have it open?
- [06:33 - 07:01] t16: So mine was open constantly. It was only when you were in a flow where you were writing a lot of code, and then you zoomed back and looked a little. Then I saw out of the corner of my eye that there are some of the arrows here that are really funky. So I think if I didn't have it open, I would forget it was there. Or if we weren't supposed to focus on it here too.
- [07:02 - 07:10] r01: Yes, because now we've punked you to use it. So you think that's mostly why it's just allowed to stay open.
- [07:11 - 07:28] t15: In my case, due to lack of space on the screen, I only brought it up after I had implemented the tasks. And used it for debugging where the errors were. So it was at the end of the flows that it was used.
- [07:28 - 07:53] t17: I think for me too, it was just like this, now I've implemented it, what are some things I have to go through. I know it explicitly says, you can't make violations, then I'll run ArchLens again. It also says, remember to build and remember to run tests, it's like just three steps. Now I've done this, I think it's finished, I go in and look, ArchLens says errors, then I'm not finished, then I have to go back again. What will it do if I say like this, now I've implemented something and run the tests, it fails, then I have to go back again.
- [07:53 - 07:54] r01: So it's just like a pipeline?
- [07:55 - 08:08] t17: Yes, in my head it was just a pipeline, because I closed it down every time, but then opened it up again. It was also lucky that it came relatively quickly. I think on larger projects I wouldn't have closed it down every time, because if it took 10 minutes to set it up, I would get tired.
- [08:10 - 08:25] t16: I also think that when I had it open all the time, I don't think you've experienced that. Every time you made some changes, it started to pop up and reload right now. And I also think that it would take time, when it's all a larger project.
- [08:27 - 08:39] r01: Okay. Were there any times when you chose not to use ArchLens, or when it was more disruptive than it was actually useful?
- [08:41 - 09:10] t17: Well, I think lack of space was a problem. So generally during development I didn't feel like I needed to use it for anything. It's not because it benefited me in terms of development in terms of whether the code I'm writing is correct. Now I'm really stuck on that logger. Is my logger correct? It can't tell me that, but it can tell me that I've made a dependency violation. So that was something I wanted to look into afterwards. But I didn't feel it benefited real development for me in terms of functionality.
- [09:12 - 09:14] r01: Do you have anything to add?
- [09:14 - 09:26] t16: No, it was mostly when you were finished and did that mental double-check, that yes, okay, there are some there.
- [09:28 - 09:37] r01: Did it change your way of thinking about the dependency policy, or was it more of a confirmation of what you already knew?
- [09:38 - 09:46] t15: More of a confirmation. It seems a bit like a kind of support wheel that you have, so that you make sure that you are always on track.
- [09:56 - 10:03] t16: Yes, it's really much the same, that it helps you to be on track. At least not to break it too much.
- [10:06 - 10:11] r01: Okay. So it didn't change the way you perceived the dependency policy?
- [10:14 - 10:33] t17: No. There was such an understanding that it was there, and it must not be broken. But I didn't feel that the visualization changed anything for me. Other than that it's just that you've made a mistake. It's not like that's why we're doing it. It explains it, or gives an overview, or gives arguments for or against, it's just that it's there.
- [10:34 - 10:54] t16: Yes. I also think in the second one, it was like, we're going to make a manager, it makes sense to move it down into infrastructure, okay no it's connected to a whole lot up, and it has to be up there, so logically I would move it somewhere else because I don't think it worked as part of the application logic. But I could see it like this if you move it over there, then we just start having all sorts of things pointing...
- [11:01 - 11:29] t17: Of course, it's also the case that you get into a situation where it's like this, the structure that has been made is just like that, if you're not used to working with it, it just creates some problems. You think one way in terms of logic in terms of what you're used to, I think another way, he thinks a third way in terms of where we want files to go with this functionality. So it's just, I think it's difficult when you just see something and you have to keep this, and then you're like okay

shit, good luck.

[11:29 - 11:40] r01: So the reason you reacted to the dependencies that ArchLens showed, which you thought were wrong, was because it was described in the README, or what?

[11:42 - 12:12] t16: Yes, that it said that you should preferably keep application and infrastructure separate from each other. So that would be the focus on where I wanted, okay, this seems more like, what should we call it, manager, or something like that, it didn't seem like a part that I wanted to place there. But I could then see visually that like, okay there's something here, where like that, I could... First I looked at, can I pull it out, so I can actually split it up, no I can't, they're connected, so, so it has to be moved up there.

[12:14 - 12:40] t17: For me, it's just a mindset where I look at it and think that's how it's set up. That's what has been decided. Fine, let's go with it. It's not because it set off any big thoughts in terms of what I would rather have. It says, that's what we're going to do, then that's what we do. It's if you're told, remember, we're doing this on this project. Received. Then that's what we do, without questioning it, if you don't have any more experience with it.

[12:43 - 12:45] r01: Would you use something like that on a real project?

[12:51 - 13:13] t15: Maybe in the initial stage of the project, where you're setting it all up, to make sure that the core structure of the project is correct. Because as soon as that part is in place, you should stay within the framework afterwards.

[13:13 - 13:14] r01: So you're thinking of some onboarding?

[13:14 - 13:33] t15: Yes, some onboarding. I also think of software architects when they have to set, the part they help set, the structure for a certain project. It could be great to use there, or also for the developers who are then allowed to start on the project after the structure has been set.

[13:35 - 14:00] t16: I think that if we look at ours, I don't think I would use it because we have a lot... A lot of it was made a bit spaghetti at the beginning, and now they've started trying to make architecture over it. I think if we started it up, we would have a lot that followed the architecture, and then a lot of parts that also didn't really do because that's what they're doing.

[14:00 - 14:01] t17: But won't it also be useful?

[14:02 - 14:10] t16: Yes, but probably mostly for the architects who are dealing with it right now. When you as a developer are dealing with it, it looks like a big spaghetti when you look at it that way.

[14:11 - 14:20] r01: So you think it's two different jobs? There should be an architect on the inside who takes things in, but the developers shouldn't really follow?

[14:21 - 14:39] t16: Not until it's in order. Because if you take it at a time when there's spaghetti in it, then you want to do your best to make it fit into the spaghetti, where once there's a structure over it, you can see the structure and follow it and say: It makes sense for us to do this.

[14:39 - 14:59] r01: Can I just, also following up on what you said, because you say that it was very good in the beginning for architects to set it up and things like that, but you used it yourself as a kind of pipeline structure to kind of look every time you had implemented something.

[15:01 - 15:18] t15: Well, yes, but I also think that in connection with onboarding, because I'm not familiar with the project, it's a good way for me to become familiar with it and see where the different boundaries are.

[15:18 - 15:23] r01: Yeah, okay, I understand. You're going to say something else, aren't you?

[15:28 - 15:29] t16: He shouldn't have anyway.

[15:30 - 15:38] r01: No. What do you say? What if you wanted to use it in a real project?

[15:38 - 17:20] t17: Well, it definitely gives an overview, especially as t15 says, newbies can really benefit from it. In relation to something like that, hey, when you do this. There are many times when it's something like that, for example, I have to make, what's it called, a handler, we have to make a handler. Well, but we have a handler folder up there, and that handler folder can only point down here and it can only point down here. So the easiest way to find out and try to get the developer to check it themselves is through a tool like this. Now let's see it quickly, you can see there's an arrow here, then you've made a terrible mistake in relation to what you've done. And that also makes it easier, I think it will also be easier when you do something like peer review. If someone is doing a PR, then it's much easier to catch. If someone has broken it by doing, that is, running this analysis, and then having it in a pipeline, where a picture just comes up and says, now the structure looks like this. So with green and red arrows you could easily use it to be a bit like, hey, we can see on your PR that you've added this green arrow, so it shouldn't be there. I think that could be useful there. It's just a question of using more as validation that it hasn't broken anything. But I also think, as t16 says, that there should be some structure beforehand. If there is no structure, and then a new arrow comes here and there, no one will be able to catch it anyway. But if it is neat, like it was here for example, where you see that you have an arrow that goes this way, and you have one that goes that way, there is no more. Then it is much more useful than if it is something like this, you have 13,000 folders and 12,000 arrows, and then it is kind of luck to find out whether the one arrow you added goes against something or not. So it depends a lot on how much structure there is on the existing project, I think, before I would use it.

[17:20 - 17:28] r01: Okay. Do you have any other comments? Anything I haven't asked about? That you think I should know?

[17:37 - 17:40] t17: Not immediately. Not something I'm thinking about.

[17:41 - 17:46] r01: Okay, I'll turn it off.